9

Introduction to Bug Bounty and Effective Reconnaissance

Chapter Objectives

After reading this chapter and completing the exercises, you will be able to do the following:

- Understand the fundamental concepts and significance of bug bounty programs in enhancing the security posture of organizations
- Identify the different types of bug bounty programs and how to participate in them
- Understand legal and ethical considerations while participating in bug bounty programs and conducting reconnaissance activities
- Develop a solid foundation in reconnaissance techniques, which is a vital skill for ethical hackers
- Use different tools and methods to conduct effective reconnaissance to identify vulnerabilities
- Apply critical thinking and analytical skills to analyze potential threats and vulnerabilities discovered during reconnaissance
- Integrate AI technologies in enhancing the efficiency and effectiveness of reconnaissance processes
- Develop strategies to effectively communicate findings to organizations, helping them remediate vulnerabilities and strengthen their security measures
- Gain practical experience through hands-on exercises designed to simulate real-world scenarios encountered in bug bounty programs and reconnaissance operations

Understanding Bug Bounty Programs

A *bug bounty* is a program initiated by companies, government agencies, and other organizations in which they allow cybersecurity experts and ethical hackers to find and report vulnerabilities present in their system, software, or application. In return for reporting these vulnerabilities, the individuals may receive rewards in the form of monetary compensation,

merchandise, or recognition. These programs aim to uncover potential security threats that might have been missed during the development phase, thus enhancing the overall security of the system and protecting it from malicious attacks.

Bug bounty programs have evolved to become a diverse platform where individuals from different backgrounds and skill levels participate. The participants in these programs can be seasoned ethical hackers and penetration testers, hobbyists, and even students. Some individuals venture into the bug bounty domain fueled by curiosity and a passion for cybersecurity. Sometimes their self-taught skills and persistent efforts result in significant contributions to the bug bounty programs. Other bug bounty hunters may have multiple years of experience in ethical hacking.

A Personal Note

Sometimes people ask me (Omar) how someone can get started in bug bounties. I find these moments to be a vivid reminder of the ever-present opportunities to mentor and nurture the next generation of cybersecurity professionals. I created the bug bounty policy at Cisco and mentored many engineers in the Product Security Incident Response Team (PSIRT). Throughout this book and chapter, I eagerly share insights and experiences, allowing you to learn from real-world experiences and scenarios.

There are several benefits of incorporating a bug bounty program in an organization:

- **Security Enhancement**: Of course, the purpose of bug bounty programs is to help uncover vulnerabilities that might have been overlooked during the development phase or by internal security teams. Traditional penetration testing is a point-in-time effort. Bug bounty programs provide a continuous assessment (more on this later in the chapter).
- **Cost-Effectiveness**: These programs can be more cost-effective than hiring a large full-time internal security team.
- **Diverse Set of Skills**: Bug bounty programs attract individuals with many different skills. Some participants may be experts in API hacking and others in vulnerabilities such as SQL injection, insecure direct object reference (IDOR) vulnerabilities, and the like. Imagine having to hire hundreds of people with these skills. Well, you can't.

- **Community Engagement**: These programs foster a community of hackers focused on enhancing security.

There are, of course, several challenges:

- **Quality Control**: Sometimes the volume of reports can contain low-quality or irrelevant submissions.
- **Management Complexity**: Managing the program and collaborating with a vast number of hackers can be complex.
- **Potential for Exploits**: If the program is not managed well, there's a risk of vulnerabilities being exploited instead of being reported.

Individuals who are starting to explore, participate in, or create bug bounty programs often ask: "What are the options?" In the next section, we explore the different types of bug bounty programs.

Types of Bug Bounty Programs

Bug bounty programs can be generally classified into public and private bug bounties. Public programs are open to all security researchers and ethical hackers across the globe. The rules and scope are publicly available, allowing anyone to participate. Private programs are invitation-only, where selected hackers are chosen based on their skills and previous performance in public programs or other platforms. **Table 9-1** shows a high-level comparison of private versus public bug bounty programs.

Table 9-1 High-Level Comparison of Private vs. Public Bug Bounty Programs

Feature	Private Bug Bounty Programs	Public Bug Bounty Programs
Accessibility	By invitation only, often extended to experienced or specialized researchers.	Open to all, allowing anyone with the necessary skills to participate.
Community Engagement	Limited to a select group of researchers/hackers.	Broader community engagement and a more diverse group of participants.

Feature	Private Bug Bounty Programs	Public Bug Bounty Programs
Reward System	Potentially higher rewards due to the exclusivity and targeted expertise required.	Reward structure might vary widely.
Confidentiality	Higher control over information dissemination and security protocols due to limited access.	Although security measures are in place, the public nature might lead to quicker exposure of vulnerabilities.
Reporting and Feedback	Direct and potentially quicker feedback due to the limited number of participants.	Structured but potentially more time-consuming reporting process due to higher volume of submissions.
Skill Sets	Potentially narrower skill set due to the number of participants.	Larger number of participants with different skills.

Clear Bug Bounty Programs

Bug bounty platforms like HackerOne offer a “clear” program where only individuals who have undergone and cleared background checks are permitted to take part in the bug bounty. This additional scrutiny is mandated due to the highly sensitive nature of the systems involved. Only hackers who have met these stringent background check criteria can participate in these exclusive programs.

Which one to use? Often, organizations at the early stages of developing a bug bounty initiative opt for private bug bounties as their preferred choice. This choice can be attributed to the controlled environment that private bounties offer, where access is limited to a select group of vetted

and experienced researchers. This choice not only ensures a focused and efficient approach to identifying vulnerabilities but also allows you to learn about scope, support, and overall cost. You have a more conducive environment for thorough and meticulous rectification work before possibly transitioning to a public bounty program in the future.

You can start a bug bounty program yourself or leverage bug bounty platform providers like HackerOne and Bugcrowd. Platform-based programs are facilitated by companies that connect hackers with organizations. Self-hosted programs are run independently by an organization, without the involvement of third-party platforms. **Table 9-2** shows a high-level comparison between platform-based and self-hosted bug bounty programs.

Table 9-2 High-Level Comparison of Platform-Based vs. Self-Hosted Bug Bounty Programs

Feature	Platform-Based Programs	Self-Hosted Bug Bounty Programs
Setup and Management	Third-party platforms handle the setup and management. Streamlined processes facilitated by platform expertise.	Managed internally by the organization. Potentially more effort required for setup and management.
Community Engagement	Larger community of security researchers and ethical hackers.	May have a smaller and often more focused community of security researchers and bug hunters. Engagement might be limited.
Reward System	Established reward systems potentially including leaderboards and reputation scores. Easier to manage payouts and	Flexibility in creating a customized reward system. Can create inconsistent reward schemes and recognitions.

Feature	Platform-Based Programs	Self-Hosted Bug Bounty Programs
	bounties.	
Security and Confidentiality	Platforms might offer standardized processes and tools for security and confidentiality. May offer additional support.	Complete control over data. However, this requires for the in-house team to manage the security processes and tools.
Reporting and Feedback	Structured reporting tools for tracking and feedback. Automated tools for analytics and insights.	You need to develop reporting formats and feedback tools.
Legal and Ethical Considerations	Platforms may provide legal guidance and frameworks. Clear guidelines for responsible disclosure.	Organizations handle legal and ethical considerations internally.

The following are a few examples of bug bounty platform providers:

- **Bugcrowd**: **https://bugcrowd.com**
- **HackerOne**: **https://hackerone.com**
- **Intigriti**: **https://intigriti.com**
- **Synack**: **https://www.synack.com**
- **YesWeHack**: **https://yeswehack.com**
- **Inspectiv**: **https://www.inspectiv.com**
- **Cobalt**: **https://cobalt.io**
- **Coder Bounty**: **http://www.coderbounty.com**
- **Bugbountyjp**: **https://bugbounty.jp**

Public programs are typically used by companies that are confident in their security posture and can handle the influx of reports from a global pool of researchers. Open-source platforms often rely on public bug

bounty programs because they encourage community involvement and transparency in securing their codebases.

Private bug bounty programs are invite-only (only selected researchers can participate). These programs are typically used when the company wants to limit access to sensitive systems or when they want more control over who is testing their products.

Often organizations that are starting with bug bounties opt for private bug bounty programs to make sure that they have the correct capacity to process bug bounty reports. They need trusted researchers who have been vetted for reliability and expertise. Private bug bounties offer more control over who participates, ensuring high-quality submissions from reputable researchers.

Tip

You can find additional bug bounty platforms and information at my GitHub repository at **https://hackerrepo.org**.

Attack Surface Management vs. Bug Bounties

Attack surface management (ASM) is a proactive security approach that involves the systematic identification, cataloging, and monitoring of all the applications, assets, and endpoints that an organization has or is connected to. This approach applies to on-premises and cloud-based applications and services. ASM is an ongoing process, undertaken by internal security teams and possibly supplemented by bug bounties and external consultants; it comprehensively includes the analysis of web domains, applications, databases, networks, and other critical infrastructures.

Attack surface management and bug bounties are two strategies that can work together to enhance the security posture of an organization. However, they have different focuses and approaches. **Table 9-3** provides a high-level comparison between attack surface management and bug bounty programs, highlighting their distinct approaches, objectives, and processes.

ASM tools automatically scan the Internet-facing assets of an organization, including domains, subdomains, IP addresses, APIs, and cloud services. They help organizations build a comprehensive inventory of all assets that could be potential entry points for attackers. Many ASM tools in-

corporate automated threat detection mechanisms that simulate attacker behavior to identify potential weaknesses before they can be exploited.

ASM solutions often integrate with other security platforms (such as SIEMs, XDR, and vulnerability management systems) to provide a unified view of the attack surface and streamline workflows. Bug bounty programs rely on human expertise to discover complex or hidden vulnerabilities that automated tools might miss. ASM tools complement bug bounties because they provide continuous visibility into an organization's attack surface.

Table 9-3 Comparing ASM and Bug Bounties

Aspect	Attack Surface Management	Bug Bounty Programs
Approach	Proactive	Reactive
Objective	Identify and mitigate vulnerabilities before exploitation.	Identify vulnerabilities through external input.
Scope	Comprehensive; includes all assets and endpoints.	Defined; focuses on specific applications or systems.
Involvement	Conducted by internal security teams, possibly with consultants.	Involves external community of ethical hackers/researchers.
Tools and Techniques	Utilizes structured tools for vulnerability scanning, and so on.	Diverse techniques by participants offering varied insights.
Frequency	Ongoing with regular assessments.	Can be periodic or ongoing with defined periods for bounties.
Policy Alignment	Aligned with organizational security policies and frameworks.	Operates based on defined rules of the bounty program.

Aspect	Attack Surface Management	Bug Bounty Programs
Response Planning	Integrated with incident response planning and security protocols.	Findings channeled into internal response processes.
Incentives	Not applicable (Internal process).	Reward-based, determined by severity and impact of findings.
Community Engagement	Limited to internal teams and consultants.	High, encourages community participation and collaboration.

Vulnerability Disclosure Programs (VDPs) vs. Bug Bounties

A vulnerability disclosure program (VDP) is a structured process established by an organization to allow security researchers, ethical hackers, and sometimes the general public to report security vulnerabilities or weaknesses that they have identified in the organization's digital assets. The primary objective of a VDP is to facilitate the responsible disclosure of vulnerabilities, confirming that identified issues are communicated to the organization in a secure and structured manner, allowing for timely mitigation before potential exploitation by malicious threat actors.

Bug bounty platforms such as Bugcrowd and HackerOne provide VDP services. These services generally outline the rules of engagement, which detail how individuals should report vulnerabilities and what kind of information should be included in the reports. VDP programs generally provide legal protection to the individuals who report vulnerabilities, safeguarding them from legal actions as long as they adhere to the guidelines stipulated in the VDP. This protection can help provide a good relationship between organizations and security researchers. VDPs often do not offer monetary rewards but may acknowledge contributors in other ways, such as featuring them in a hall of fame or providing certificates of recognition.

Getting Started: Preparing Yourself as an Ethical Hacker in Bug

Bounty Programs

If you're a beginner, let's go over a few tips and a structured approach to get started in bug bounties. The first thing is to start with the basics. You don't need to be a seasoned programmer. However, learning programming languages such as Python and JavaScript will help you understand the software's inner workings and identify vulnerabilities effectively.

Learn networking! Familiarize yourself with the basics of networking concepts, including TCP/IP, HTTP/HTTPS, DNS, and more. This knowledge will aid in understanding how data moves through networks and potentially where security loopholes might be. Understanding general cybersecurity fundamentals is vital. You might want to research deep into concepts like cryptography, web application security, API security, and network security.

Set up a lab. Create a virtual lab using tools like VirtualBox or Proxmox to set up a controlled environment where you can practice your skills without any legal consequences.

Learn with WebSploit Labs

You can start with WebSploit Labs. WebSploit Labs is a learning environment that I created for different cybersecurity ethical hacking, bug hunting, incident response, digital forensics, and threat hunting training sessions. WebSploit Labs includes several intentionally vulnerable applications running in Docker containers on top of Kali Linux or Parrot Security OS, several additional tools, and thousands of cybersecurity resources. WebSploit Labs has been used by many colleges and universities in different countries. It comes with over 500 distinct exercises.

You can also use platforms like Hack The Box (**https://hackthebox.com**), TryHackMe (**https://tryhackme.com**), and Pentester Lab (**https://pentesterlab.com**), or participate in capture-the-flag (CTF) events.

Being informed about the latest trends and advancements in the field is necessary. You can follow communities, forums, and social media channels that focus on cybersecurity.

Choose the right bug bounty platform for you, not for your neighbor or for a YouTuber. Platforms like HackerOne, Bugcrowd, and Intigriti are

popular choices where you can start your bug bounty journey. However, each has its own benefits. You can participate in multiple platforms. However, before you start hunting for bugs, ensure that you thoroughly understand the rules and guidelines set by the platform to avoid any legal issues.

When you find a vulnerability, it's vital to report that vulnerability in a structured manner, detailing the steps to reproduce the bug and potential impacts. Additionally, maintain a portfolio where you document your achievements, certifications, and the bugs you have identified. This portfolio will serve as a testament to your skills and experience.

Tip

You can find a collection of real-life bug bounty reports and walkthroughs at my GitHub repository by going to **https://hackerrepo.org** and navigating under **bug-bounties**. You should also develop soft skills like clear communication and problem-solving, which are also important in becoming successful and communicating your findings effectively.

Understanding the Scope and Rules of Engagement

Understanding the scope within bug bounty programs is a critical element, often serving as the foundation upon which the success and efficacy of the program are established. The scope of a bug bounty program outlines the specific parameters within which hackers are permitted to operate, thus delineating clear boundaries and expectations. This delineation is extremely important for both ethical hackers and organizations creating a bug bounty program.

In the context of a bug bounty program, the scope typically includes details such as the domains that can be tested, the types of vulnerabilities that are sought, and the technologies that are used in the infrastructure. Additionally, it clearly defines what is off-limits, preventing unintentional overreach and helping to maintain a respectful and lawful relationship between the researchers and the organization.

Beyond just a list of targets, the scope serves as a detailed guideline, helping researchers to prioritize their efforts effectively. It aids them in focusing their expertise and time on areas that are considered to be of high

value or potentially more vulnerable, thus optimizing the chances of identifying significant security flaws.

Having a well-defined scope can be instrumental in avoiding legal complications because it explicitly states the boundaries within which the hackers are allowed to operate. It serves as a contractual agreement, protecting both the organization and the researchers from potential disputes. **Table 9-4** lists the different aspects to be considered when creating a scope and rules of engagement for a bug bounty program.

Table 9-4 Defining the Scope and Rules of Engagement in Bug Bounties

Aspect	Description
Target Systems	Provide details of the specific systems, applications, or domains that are open for testing. They may include web applications, mobile apps, and APIs.
Vulnerability Types	Provide a list of specific vulnerability types that the program is interested in identifying. They could range from SQL injection and cross-site scripting (XSS) to more complex issues like server-side request forgery (SSRF) and business logic vulnerabilities.
Testing Methods	Define the acceptable methods and tools that researchers can use during their testing. It may specify limitations to prevent disruptions or damage to the live environment.
Out-of-Scope Targets	Clearly list the systems, data types, or vulnerabilities that are not a part of the program, and thus should not be targeted during the testing.
Reward Structure	Describes the reward structure, including the types of rewards (monetary, swag, recognition) and the criteria for determining the reward amounts.
Reporting Guidelines	Specify the process for reporting identified vulnerabilities, including the format, the information required, and the expected timeline for feedback and remediation.

It is very important that organizations maintain a dynamic approach to defining the scope, allowing for adjustments and expansions as the program evolves. This adaptability ensures that the program remains relevant and continues to address the changing threat landscape effectively.

Example 9-1 shows a bug bounty scope and rules of engagement template.

Example 9-1 Bug Bounty Scope and Rules of Engagement Template

Click here to view code image

```
Omar's Bug Bounty Program Scope Template

Target Systems
==================

In-Scope Targets
-----------------
- Web Applications
  - app1.websploit.org
  - app2.websploit.org

- Mobile Applications
  - Android App (version x.x and above)
  - iOS App (version x.x and above)

- APIs
  - api.websploit.org/v1/
  - api.websploit.org/v2/

Out-of-Scope Targets
--------------------
- app3.websploit.org
- partner.websploit.org

Vulnerability Types
===================

In-Scope Vulnerabilities
------------------------
- Cross-Site Scripting (XSS)
- SQL Injection
- Cross-Site Request Forgery (CSRF)
- Server-side Request Forgery (SSRF)
- Business Logic Vulnerabilities

Out-of-Scope Vulnerabilities
```

```
---------------------------
- Denial of Service (DoS) attacks
- Social Engineering Attacks

Reward Structure
================
- Critical Vulnerabilities: $10,000 - $50,000 (or alternative rewards)
- High Severity Vulnerabilities: $500 - $1000 (or alternative rewards)
- Medium Severity Vulnerabilities: $100 - $500 (or alternative rewards)
- Low Severity Vulnerabilities: $50 - $100 (or alternative rewards)
(Include criteria for determining the severity)
```

Exploring Effective Reconnaisance

The first step a threat actor takes when planning an attack is to gather information about the target. This act of information gathering is known as *reconnaissance* (or *recon*). Attackers use scanning and enumeration tools along with public information available on the Internet to build a dossier about a target. As you can imagine, as a penetration tester, you must also replicate these methods to determine the exposure of the networks and systems you are trying to defend.

Let's begin with a discussion of what's an effective reconnaissance strategy. You will briefly learn about some of the common tools and techniques used. From there, we'll dig deeper into the process of vulnerability scanning and how scanning tools work, including how to analyze the vulnerability scanner results to provide useful deliverables and explore the process of leveraging the gathered information in the exploitation phase. We will also explore some of the common challenges to consider when performing vulnerability scans.

Active vs. Passive Reconnaissance

Active reconnaissance is a method of information gathering in which the tools used actually send out probes to the target network or systems to elicit responses that are then used to determine the posture of the network or system. These probes can use different protocols and multiple levels of aggressiveness, typically based on what is being scanned and when. For example, you may be scanning a device such as a printer that might not have a very robust TCP/IP stack or network hardware. By sending active probes, you might crash such a device. Most modern devices do not have this problem; however, it is possible, so when doing active scanning, you should be conscious of this and adjust your scanner settings accordingly.

Passive reconnaissance is a method of information gathering in which the tools do not interact directly with the target device or network. There are multiple methods of passive reconnaissance. Some involve using third-party databases to gather information. Others might also use tools in such a way that they will not be detected by the target. These tools, in particular, work by simply listening to the traffic on the network and using intelligence to deduce information about the device communication on the network. This approach is much less invasive on a network, and it is highly unlikely for this type of reconnaissance to crash a system such as a printer. Because it does not produce any traffic, it is also unlikely to be detected and does not raise any flags on the network that it is surveying. Another scenario in which a passive scanner would come in handy would be for a penetration tester who needs to perform analysis on a production network that cannot be disrupted.

Common active reconnaissance methods include the following:

- Host enumeration
- Network enumeration
- User enumeration
- Group enumeration
- Network share enumeration
- Web page enumeration
- Application enumeration
- Service enumeration

Common passive reconnaissance methods include the following:

- Domain enumeration
- Packet capturing and eavesdropping
- Open-source intelligence (OSINT)

Understanding OSINT

Open-source intelligence (OSINT) refers to the process of collecting and analyzing publicly available information from different sources. In the context of bug bounty programs, OSINT serves as a powerful tool for security researchers to gather data and identify potential vulnerabilities in a target system. Leveraging OSINT can help in painting a comprehensive picture of the target's security posture, ultimately aiding in effective vulnerability identification and exploitation.

Tip

I have compiled an extensive list of OSINT tools and resources in my GitHub repository. I continuously update this repository to include the latest resources available. You can access it at **https://github.com/The-Art-of-Hacking/h4cker** or at **hackerrepo.org**. I highly recommend bookmarking this link to stay informed of the most up-to-date tools and techniques that can significantly augment your bug bounty reconnaissance activities. Another great resource is the OSINT Framework at **https://osintframework.com**. The OSINT Framework is a visualization that includes numerous OSINT-related resources, as shown in **Figure 9-1**.

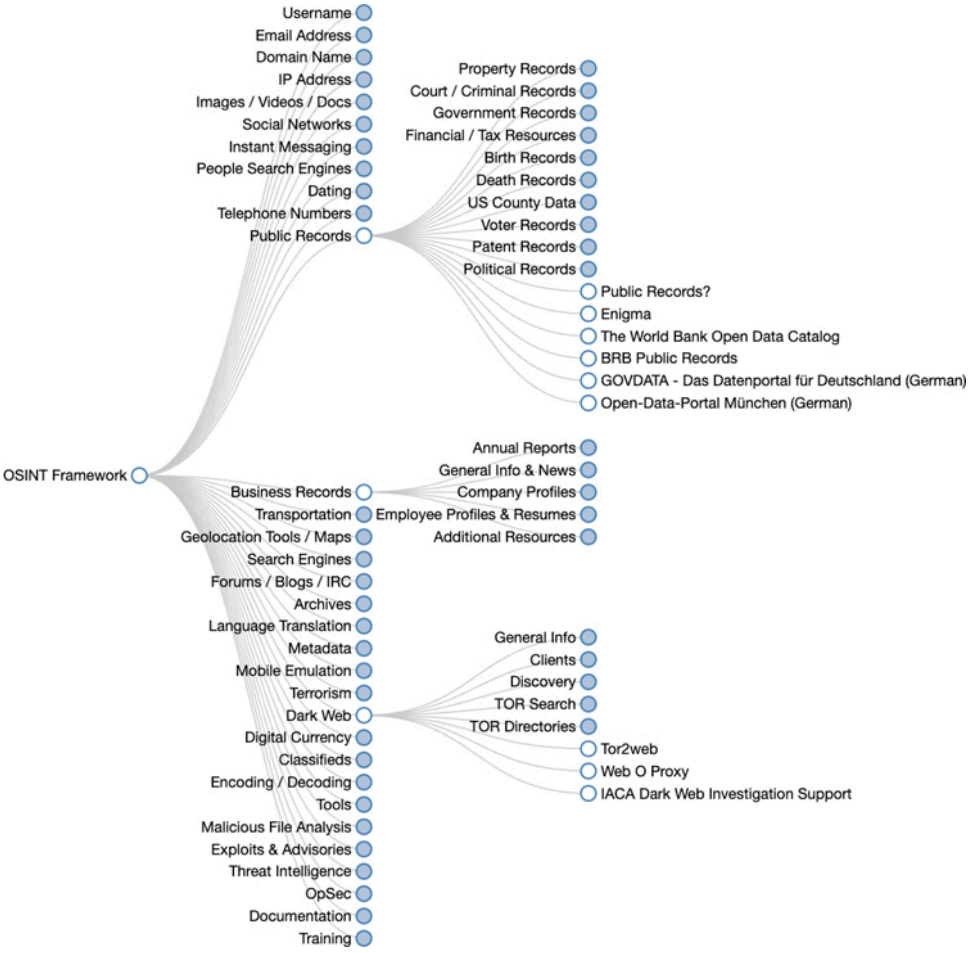


Figure 9-1

The OSINT Framework

Leveraging DNS for Recon

Suppose, for example, that an attacker has a target, **h4cker.org**, in its sights. In this case, **h4cker.org** has an Internet presence, as most companies do. This presence is a website hosted at **www.h4cker.org**. Just as a home burglar would need to determine which entry and exit points exist

in a home before being able to commit a robbery, a cyber attacker needs to determine which of the target's ports and protocols are exposed to the Internet. A burglar might walk around the outside of the house, looking for doors and windows and then possibly looking at the locks on the doors to determine their weaknesses. Similarly, a cyber attacker would perform tasks like scanning and enumeration.

Typically, an attacker would start with a small amount of information and gather more information while scanning, eventually moving on to perform different types of scans and to gather additional information. For instance, the attacker targeting **h4cker.org** might start by using *DNS lookups* to determine the IP address or addresses used by **h4cker.org** and any other subdomains that might be in use. Let's assume those queries reveal that **h4cker.org** is using the IP addresses 185.199.108.153 for **www.h4cker.org**, 185.199.110.153 for **mail.h4cker.org**, and 185.199.110.153 for **portal.h4cker.org**. **Example 9-2** shows an example of the DNSRecon tool in Kali Linux being used to query the DNS records for **h4cker.org**.

Example 9-2 DNSRecon Example

Click here to view code image

```
┌─[omar@websploit]─[~]
└── $dnsrecon -d h4cker.org
[*] Performing General Enumeration of Domain: h4cker.org
/usr/share/dnsrecon/./dnsrecon.py:816: DeprecationWarning: please use dns.resolver.
Resolver.resolve() instead
  answer = res._res.query(domain, 'DNSKEY')
[*] DNSSEC is configured for h4cker.org
[*] DNSKEYs:
[*] NSEC3 ZSK RSASHA256 030100019ed0af43a7dc09d07e1646d2 b4036075e9187c-
4c563519155f888b60 8fdffe9c6d8a0a01522f78d25d257772 0a8e97d1350e694b272e-
c63af9708609 b3721e6b53a2d7aa8839585714800319 dd98f97b39d8768f7e975a449c001ce9
55189ea83f30a4fe6b4dff7b3dd15f89 1cef3a8d84968a980bde65c0b1309d5b 825a0f23
[*] NSEC3 KSk RSASHA256 030100018403e0971df0dc1770f3b96a ca57eb68d03a84b4a712cad-
da60567fe a264f0e5d7ec4c8e0187300f0933f419 d22a17548c3a046636666300c06711f0
761200245149a220b79918b3f38a9a6e 8228425cb39b6466adba9f6f7fe28d76 c1bcf44e19f-
035f658eef65cb630638f 7aa15d7706cc572c863d65619bd48f77 425ea0844716709b9923117ade
41d414 c94f8e581db9274cf1c8bb41fbbd7838 24978c0f9b7125b9ce3e8abe442a6bc7 4bf519790
a18a27916c946f503c02b08 0a8550bc5b9b147d581a3f5f763df377 9e1d655c51c2e06aa2062d1f-
08f34abc 37947ac48403dc0da9af846c7a4caeae 7567bb8fdf625b1a179e6fd6faf35be9 09488cb9
[*] SOA ns-cloud-c1.googledomains.com 216.239.32.108
[*] NS ns-cloud-c1.googledomains.com 216.239.32.108
<output omitted for brevity>
[*] MX alt3.aspmx.l.google.com 2a00:1450:400c:c0b::1b
```

```
[*] MX alt4.aspmx.l.google.com 2a00:1450:4013:c02::1b
[*]  A h4cker.org 185.199.109.153
[*] SPF v=spf1 include:_spf.google.com ~all
[*] TXT h4cker.org v=spf1 include:_spf.google.com ~all
[*] Enumerating SRV Records
[+] 0 Records Found
┌─[omar@websploit]─[~]
└── $
```

From there, an attacker can begin to dig deeper by scanning the identified hosts. Once the attacker knows which hosts are alive on the target site, they would then need to determine what kind of services the hosts are running. To do this, the attacker might use the tried-and-true Nmap tool. Before we discuss this tool and others in depth, we need to look at the types of scans and enumerations you should perform and why. Nmap was once considered a simple port scanner; however, it has evolved into a much more robust tool that can provide additional functionality, thanks to the Nmap Scripting Engine (NSE).

You can use other basic DNS tools such as **nslookup**, **host**, and the Linux **dig** command to perform name resolution and obtain additional information about a domain. **Example 9-3** shows how the **dig** tool is used to show the DNS resolution details for **h4cker.org**.

Example 9-3 Using *dig* to Obtain Information About a Given Domain

Click here to view code image

```
┌─[omar@websploit]─[~]
└── $dig h4cker.org
; <<>> DiG 9.16.6-Debian <<>> h4cker.org
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 6517
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 1
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;h4cker.org.                    IN      A
;; ANSWER SECTION:
h4cker.org.             172     IN      A       185.199.110.153
h4cker.org.             172     IN      A       185.199.111.153
h4cker.org.             172     IN      A       185.199.108.153
h4cker.org.             172     IN      A       185.199.109.153
;; Query time: 72 msec
;; SERVER: 208.67.222.222#53(208.67.222.222)
```

```
;; MSG SIZE  rcvd: 103

┌─[omar@websploit]─[~]
└── $
```

The highlighted lines show the IP addresses associated with **h4cker.org**. Now, let's approach this scenario in a more programmatic way and see the email servers used by **websploit.org** (mail exchanger [MX] record). To do so, let's look at the Python script demonstrated in **Example 9-4**.

Example 9-4 Obtaining the MX Record of **websploit.org** Using Python

Click here to view code image

```
import dns.resolver
def get_mx_record(domain):
    try:
        result = dns.resolver.resolve(domain, 'MX')
        for rdata in result:
            print(f'MX Record: {rdata.exchange.to_text()} with priority
{rdata.preference}')
    except dns.resolver.NoAnswer:
        print(f"No MX records found for domain {domain}")
    except dns.resolver.NXDOMAIN:
        print(f"The domain {domain} does not exist")
    except Exception as e:
        print(f"An error occurred: {e}")

# Replace “websploit.org” with the domain you are interested in
get_mx_record(‘websploit.org’);; MSG SIZE  rcvd: 157
```

This script is also available in my GitHub repository at **hackerrepo.org**. Before you run the Python script in **Example 9-4**, you will need to install the dnspython library using the command **pip3 install dnspython**. In this script:

1. We first import the dns.resolver module from the dnspython library.
2. We define a function called **get_mx_record** that takes a domain name as an argument.
3. Inside the function, we use a **try-except** block to catch various potential exceptions that might occur during the DNS query.
4. We use **dns.resolver.resolve(domain, 'MX')** to query the MX records for the domain.
5. If the query is successful, we loop through each record and print the exchange and preference values.

Remember to replace **wesbploit.org** with the domain for which you want to get the MX record information.

Tip

Tools like MassDNS can help accelerate a lot of the DNS reconnaissance tasks. MassDNS is a scalable DNS stub resolver designed for individuals aiming to process a considerable number of domain names, ranging into the millions or even billions. Without requiring any specific settings, MassDNS can handle over 350,000 name resolutions per second utilizing publicly accessible resolvers. You can access MassDNS's source code at **https://github.com/blechschmidt/massdns**.

Identifying Technical and Administrative Contacts

You can easily identify domain technical and administrative contacts by using the **whois** tool. Keep in mind that many organizations keep their registration details private and instead use the domain register organization contacts. For instance, let's look at the whois details for **tesla.com** shown in **Example 9-5**.

Example 9-5 Whois Information for the Domain **tesla.com**

Click here to view code image

```
┌─[omar@websploit]─[~]
└── $whois tesla.com
   Domain Name: TESLA.COM
   Registry Domain ID: 187902_DOMAIN_COM-VRSN
   Registrar WHOIS Server: whois.markmonitor.com
   Registrar URL: http://www.markmonitor.com
   Registrar: MarkMonitor Inc.
   Registrar IANA ID: 292
   Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
   Registrar Abuse Contact Phone: +1.2083895740
   Domain Status: serverUpdateProhibited
https://icann.org/epp#serverUpdateProhibited
   Name Server: A1-12.AKAM.NET
<output omitted for brevity>
The Registry database contains ONLY .COM, .NET, .EDU domains and
Registrars.
Domain Name: tesla.com
Registry Domain ID: 187902_DOMAIN_COM-VRSN
<output omitted for brevity>
```

```
Registry Registrant ID:
Registrant Name: Domain Administrator
Registrant Organization: DNStination Inc.
Registrant Street: 3450 Sacramento Street, Suite 405
Registrant City: San Francisco
Registrant State/Province: CA
Registrant Postal Code: 94118
Registrant Country: US
Registrant Phone: +1.4155319335
Registrant Phone Ext:
Registrant Fax: +1.4155319336
Registrant Fax Ext:
Registrant Email: admin@dnstinations.com
Registry Admin ID:
Admin Name: Domain Administrator
Admin Organization: DNStination Inc.
Admin Street: 3450 Sacramento Street, Suite 405
Admin City: San Francisco
Admin State/Province: CA
Admin Postal Code: 94118
Admin Country: US
Admin Phone: +1.4155319335
Admin Phone Ext:
Admin Fax: +1.4155319336
Admin Fax Ext:
Admin Email: admin@dnstinations.com
Registry Tech ID:
Tech Name: Domain Administrator
Tech Organization: DNStination Inc.
Tech Street: 3450 Sacramento Street, Suite 405
Tech City: San Francisco
Tech State/Province: CA
Tech Postal Code: 94118
Tech Country: US
Tech Phone: +1.4155319335
Tech Phone Ext:
Tech Fax: +1.4155319336
Tech Fax Ext:
Tech Email: admin@dnstinations.com
Name Server: edns69.ultradns.org
Name Server: edns69.ultradns.net
Name Server: edns69.ultradns.com
<output omitted for brevity>
MarkMonitor reserves the right to modify these terms at any time.
By submitting this query, you agree to abide by this policy.
MarkMonitor Domain Management(TM)
Protecting companies and consumers in a digital world.
Visit MarkMonitor at https://www.markmonitor.com
```

```
Contact us at +1.8007459229
In Europe, at +44.02032062220
```

The highlighted lines in **Example 9-5** show the technical and administrative contacts for the domain (which are also the ones for the domain registrar [MarkMonitor]).

Note

The introduction of the General Data Protection Regulation (GDPR) in Europe has significantly impacted the availability and accessibility of WHOIS data, which provides public information about domain registrations. WHOIS data traditionally includes details such as the registrant's name, address, phone number, and email, which were valuable for cybersecurity research, intellectual property enforcement, and business purposes. However, GDPR's focus on protecting personal data has led to substantial changes in how this information is handled by most companies.

Example 9-6 shows another example (the technical and administrative email contacts for the domain **cisco.com**).

Example 9-6 Showing Technical and Administrative Email Contacts

Click here to view code image

```
┌─[omar@websploit]─[~]
└── $whois cisco.com | grep '@cisco.com'
Registrant Email: infosec@cisco.com
Admin Email: infosec@cisco.com
Tech Email: infosec@cisco.com
```

In **Example 9-6**, the technical and administrative contacts point to the InfoSec team at Cisco (infosec@cisco.com) instead of the registrant.

Tip

Tools such as **recon-ng, theharvester, maltego,** and others help automate the process of passive reconnaissance and support many DNS-based and whois queries. Several of these tools are listed in my GitHub repository at

https://github.com/The-Art-of-Hacking/h4cker/tree/master/osint.

Identifying Cloud vs. Self-Hosted Assets

A company may own a domain and related subdomain, but its applications may be hosted in the cloud. For example, Netflix (at the time of writing) owns the domain **netflix.com**, which resolves to IPv4 addresses 3.230.129.93, 52.3.144.142, and 54.237.226.164 (as demonstrated in **Example 9-7** with the Linux **host** command).

Example 9-7 DNS Name Resolution for **netflix.com**

Click here to view code image

```
┌─[omar@websploit]─[~]
└── $host netflix.com
netflix.com has address 3.230.129.93
netflix.com has address 52.3.144.142
netflix.com has address 54.237.226.164
netflix.com has IPv6 address 2600:1f18:631e:2f80:77e5:13a7:6533:7584
netflix.com has IPv6 address 2600:1f18:631e:2f82:c8cd:27b2:ac:8dbf
netflix.com has IPv6 address 2600:1f18:631e:2f84:ceae:e049:1e:6a96
netflix.com mail is handled by 1 aspmx.l.google.com.
netflix.com mail is handled by 5 alt1.aspmx.l.google.com.
netflix.com mail is handled by 5 alt2.aspmx.l.google.com.
netflix.com mail is handled by 10 aspmx2.googlemail.com.
netflix.com mail is handled by 10 aspmx3.googlemail.com.
```

However, the IPv4 addresses 3.230.129.93, 52.3.144.142, and 54.237.226.164 are owned by Amazon, and **netflix.com** is hosted in Amazon Web Services (AWS), as demonstrated in **Example 9-8**.

Example 9-8 The Ownership of IP Addresses and Applications Hosted in the Cloud

Click here to view code image

```
┌─[omar@websploit]─[~]
└── $whois 3.230.129.93 | grep OrgName
OrgName:        Amazon Technologies Inc.
OrgName:        Amazon Data Services NoVa
┌─[omar@websploit]─[~]
└── $whois 52.3.144.142 | grep OrgName
```

```
OrgName:        Amazon Technologies Inc.
┌─[omar@websploit]─[~]
└── $whois 54.237.226.164 | grep OrgName
OrgName:        Amazon Technologies Inc.
OrgName:        Amazon.com, Inc.
```

In **Example 9-8**, the **whois** command is used to retrieve the organization name (OrgName) of the owner for each of the IP addresses (3.230.129.93, 52.3.144.142, and 54.237.226.164).

Now, let's create a Python script that can do this for us very conveniently. **Example 9-9** shows a Python script that takes a domain as an argument, performs a DNS lookup to get its IP address, and then performs a WHOIS lookup to determine if the IP address belongs to a major cloud provider. We will use the **socket** module for DNS lookup and **python-whois** package for the WHOIS lookup. You will need to install the **python-whois** package to run this script. You can do this by running **pip install python-whois**.

Example 9-9 Using Python: Cloud Checker Script

Click here to view code image

```
# Import the necessary modules
import socket
import whois

def get_domain_ip(domain):
    '''
    get domain IP address
    :param domain: domain
    :return: ip_address
    '''
    try:
        ip_address = socket.gethostbyname(domain)
        return ip_address
    except socket.gaierror:
        return None

def get_whois_info(ip_address):
    '''
    get whois information
    :param ip_address: ip_address
    :return: whois information
    '''
    try:
```

```
        w = whois.whois(ip_address)
        return w
    except whois.WhoisException:
        return None

def is_major_cloud_provider(whois_info):
    '''
    check if the IP address belongs to a major cloud provider
    :param whois_info: whois_info
    :return: True or False
    '''
    cloud_providers = ["Amazon", "Azure", "Microsoft", "Google", "Digital Ocean",
"Alibaba", "Oracle"]
    for provider in cloud_providers:
        if provider.lower() in whois_info.lower():
            return True
    return False

def main(domain):
    '''
    main function
    :param domain: domain
    :return: None
    '''
    ip_address = get_domain_ip(domain)
    if ip_address:
        print(f"The IP address of {domain} is {ip_address}")
        whois_info = get_whois_info(ip_address)
        if whois_info and whois_info.org:
            print(f"The organization owning the IP is: {whois_info.org}")
            if is_major_cloud_provider(str(whois_info.org)):
                print(f"The IP address belongs to a major cloud provider.")
            else:
                print(f"The IP address does not belong to a known major cloud
provider.")
        else:
            print(f"Could not retrieve WHOIS information.")
    else:
        print(f"Could not find IP address for {domain}.")

if __name__ == "__main__":
    domain = input("""Cloud Checker example by Omar Santos.
Please enter a domain: """)
    main(domain)
```

The script in **Example 9-9** is also available at the GitHub repository at **https://github.com/The-Art-of-Hacking/h4cker/tree/master/programming_and_scripting_for_cybersecurity/recon_scripts/dns**

or by visiting **hackerrepo.org**. **Example 9-10** demonstrates how to use the script.

Example 9-10 Using the Cloud Checker Script

Click here to view code image

```
┌──(root㉿websploit)-
[~/h4cker/programming_and_scripting_for_cybersecurity/recon_scripts/dns_recon]
└─# python3 cloud_provider.py
Cloud Checker example by Omar Santos.
Please enter a domain: netflix.com
The IP address of netflix.com is 3.230.129.93
The organization owning the IP is: Amazon.com, Inc.
The IP address belongs to a major cloud provider.

┌──(root㉿websploit)-
[~/h4cker/programming_and_scripting_for_cybersecurity/recon_scripts/dns_recon]
└─# python3 cloud_provider.py
Cloud Checker example by Omar Santos.
Please enter a domain: cisco.com

The IP address of cisco.com is 72.163.4.185
The organization owning the IP is: Cisco Technology Inc.
The IP address does not belong to a known major cloud provider.
```

Social Media Scraping

Attackers can easily gather valuable information about their victims by scraping social media sites such as X (previously known as Twitter), LinkedIn, Facebook, Instagram, and others. People post too many things online. They publicly talk about their hobbies, what restaurants they visit, what they do for work, if they got a promotion, where they travel for business and pleasure, and much more.

Often attackers use other information such as *key contacts* (company stakeholders) and their *job responsibilities*. Attackers can use all that information to perform different types of social engineering attacks (including spear phishing and whaling). You learned details about social engineering attacks in **Chapter 5**, "**Social Engineering Attacks and Physical Assessments**."

Attackers often leverage *job listings* in websites like Indeed, LinkedIn, Career Builder, and individual company websites to obtain information about what technologies each company is using (that is, the *technology*

stack of the organization). Let's say that the company is looking for a Cisco firewall administrator, a LangChain expert, and a Mongo database architect in Raleigh, North Carolina. The attacker doesn't have to launch any tools to learn that the company is using Cisco firewalls, Mongo databases, and probably using LangChain for AI-driven automation in the Raleigh office.

Similarly, attackers have also created job posts to attract people to apply for those positions. Then they interview their victims to try to get them to talk about what they do at work and probably the technologies used by their employer. Think about it. When people are trying to get a job, they want to show off and, as a result, often reveal too much information about the technologies, architectures, and applications that they use in their current role.

Cryptographic Flaws

During the reconnaissance phase, attackers often can inspect *secure socket layer (SSL) certificates* to obtain information about the organization, potential *cryptographic flaws*, and weak implementations. What can you find inside of digital certificates? The certificate serial number, subject common name, the URI of the server it was assigned to, the organization name, the online certificate status protocol (OCSP) information, the certificate revocation list (CRL), URI, and so on.

Note

Certificate *revocation* is the act of invalidating a digital certificate. For instance, if an application has been decommissioned or the certificate assigned to such application is compromised, you should revoke the certificate and add its serial number to a CRL. OCSPs and CRLs are used to verify whether a certificate has been revoked (that is, invalidated by the issuing authority).

Now, let's look at the digital certificate assigned to **websploit.org** shown in **Example 9-11**. This certificate shows the organization that issued the certificate (in this case Let's Encrypt [letsencrypt.org]), the serial number, validity period, and public key information (including the algorithm and key size). Attackers can use this information to reveal any weak cryptographic configuration or implementation.

Example 9-11 Obtaining the Certificate Information from a Website Using OpenSSL

Click here to view code image

```
$ openssl s_client -connect websploit.org:443 -servername websploit.org
2>/dev/null | openssl x509 -text -noout
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            03:c3:a1:8f:c6:d7:88:53:9c:0b:fe:e1:6e:58:5a:4f:64:36
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C = US, O = Let's Encrypt, CN = R3
        Validity
            Not Before: Aug 15 22:14:30 2024 GMT
            Not After : Nov 13 22:14:29 2024 GMT
        Subject: CN = websploit.org
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Modulus:
                    00:ab:c7:1b:0c:ed:c6:01:f8:ea:a9:b3:cf:08:17:
                    4f:a2:cb:7c:34:c4:66:12:e6:ef:f3:98:17:79:c9:
                    65:ee:66:4c:1f:9a:92:7d:33:ee:07:fa:2e:15:62:
<output omitted for brevity>
                Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Key Usage: critical
                Digital Signature, Key Encipherment
            X509v3 Extended Key Usage:
                TLS Web Server Authentication, TLS Web Client Authentication
            X509v3 Basic Constraints: critical
                CA:FALSE
            X509v3 Subject Key Identifier:
                63:4E:15:85:56:5A:A4:94:02:C2:16:42:A4:A5:97:9A:38:02:57:97
            X509v3 Authority Key Identifier:
                14:2E:B3:17:B7:58:56:CB:AE:50:09:40:E6:1F:AF:9D:8B:14:C2:C6
            Authority Information Access:
                OCSP - URI:http://r3.o.lencr.org
                CA Issuers - URI:http://r3.i.lencr.org/
            X509v3 Subject Alternative Name:
                DNS:websploit.org
            X509v3 Certificate Policies:
                Policy: 2.23.140.1.2.1
            CT Precertificate SCTs:
                Signed Certificate Timestamp:
                    Version   : v1 (0x0)
```

```
                    Log ID    : B7:3E:FB:24:DF:9C:4D:BA:75:F2:39:C5:BA:58:F4:6C:
                                5D:FC:42:CF:7A:9F:35:C4:9E:1D:09:81:25:ED:B4:99
                    Timestamp : Aug 15 23:14:30.605 2023 GMT
                    Extensions: none
                    Signature : ecdsa-with-SHA256
                                30:46:02:21:00:8E:EE:71:7F:65:2C:36:FF:88:49:0F:
                                0D:67:9F:B8:44:E3:57:24:19:CB:94:AB:68:C0:0A:FB:
                                CB:AB:7A:7D:5D:02:21:00:CD:BC:B7:0F:3F:83:26:AB:
                                16:A8:E3:D8:CD:9D:C5:EA:AB:62:11:94:6E:AD:51:6B:
                                4B:0F:57:34:A0:84:32:9C
                Signed Certificate Timestamp:
                    Version   : v1 (0x0)
                    Log ID    : 7A:32:8C:54:D8:B7:2D:B6:20:EA:38:E0:52:1E:E9:84:
                                16:70:32:13:85:4D:3B:D2:2B:C1:3A:57:A3:52:EB:52
                    Timestamp : Aug 15 23:14:30.644 2023 GMT
                    Extensions: none
                    Signature : ecdsa-with-SHA256
                                30:44:02:20:6F:55:96:EA:54:27:76:D8:A2:37:1A:F0:
                                9A:E4:A7:50:5A:C3:60:F6:F7:A2:8A:F0:28:49:21:01:
                                2A:0A:3F:E3:02:20:78:A2:EC:95:30:D6:0E:35:42:BD:
                                0F:30:4F:B9:8C:79:6B:5C:9E:F5:31:AF:29:0E:C2:A7:
                                31:D1:71:14:36:0E
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        56:27:a3:e0:8a:7e:18:85:23:04:84:73:f4:63:53:f5:4d:33:
        02:3e:02:cb:c1:1e:ea:9b:33:01:c0:5c:05:36:8d:77:ca:75:
        a4:21:f6:ac:ea:07:7f:76:dc:10:de:c2:da:ff:6f:6d:71:b7:
        63:61:f6:6a:8d:ea:3e:38:90:2b:56:e9:12:3a:06:43:53:c6:
```

In **Example 9-11**, the **openssl s_client -connect websploit.org:443 -servername websploit.org** part of the command connects to the server and retrieves the certificate. The **2>/dev/null** part discards error messages, if any, and **openssl x509 -text -noout** pipes the certificate output from the previous command to another **openssl** command that prints the certificate details in a readable format. You can replace **websploit.org** with the actual domain from which you want to retrieve the certificate information.

Several tools can be used to reveal weak crypto implementations in websites. One of the most popular is **testssl.sh**, which can be installed by using the **apt install testssl.sh** command in WebSploit Labs (running Kali or Parrot) or by following the instructions at **https://testssl.sh**. **Example 9-12** shows how to run this tool against the **websploit.org** domain.

Example 9-12 Detecting Weak Crypto Implementations

Click here to view code image

```
┌──(root㉿websploit)-[~]
└─# testssl websploit.org

<output omitted for brevity>
Testing all IPv4 addresses (port 443): 185.199.109.153 185.199.108.153
185.199.111.153 185.199.110.153
--------------------------------------------------------------------------------
----------------
 Start 20:09:38        -->> 185.199.109.153:443 (websploit.org) <<--
 Further IP addresses:   185.199.108.153 185.199.110.153 185.199.111.153
 rDNS (185.199.109.153): cdn-185-199-109-153.github.com.
 Service detected:       HTTP
 Testing protocols via sockets except NPN+ALPN
 SSLv2      not offered (OK)
 SSLv3      not offered (OK)
 TLS 1      not offered
 TLS 1.1    not offered
 TLS 1.2    offered (OK)
 TLS 1.3    offered (OK): final
 NPN/SPDY   not offered
 ALPN/HTTP2 h2, http/1.1 (offered)

 Testing cipher categories
 NULL ciphers (no encryption)                  not offered (OK)
 Anonymous NULL Ciphers (no authentication)    not offered (OK)
 Export ciphers (w/o ADH+NULL)                 not offered (OK)
 LOW: 64 Bit + DES, RC[2,4] (w/o export)       not offered (OK)
 Triple DES Ciphers / IDEA                     not offered
 Obsolete CBC ciphers (AES, ARIA etc.)         offered
 Strong encryption (AEAD ciphers)              offered (OK)

Testing robust (perfect) forward secrecy, (P)FS -- omitting Null Authentication/
Encryption, 3DES, RC4
 PFS is offered (OK)          TLS_AES_256_GCM_SHA384 TLS_CHACHA20_POLY1305_SHA256
ECDHE-RSA-AES256-GCM-SHA384 ECDHE-RSA-AES256-SHA384 ECDHE-RSA-AES256-SHA ECDHE-RSA-
CHACHA20-POLY1305 TLS_AES_128_GCM_SHA256 ECDHE-RSA-AES128-GCM-SHA256 ECDHE-
RSA-AES128-SHA256 ECDHE-RSA-AES128-SHA
 Elliptic curves offered:     prime256v1 secp384r1 secp521r1 X25519 X448
 Testing server preferences
 Has server cipher order?     yes (OK) -- only for < TLS 1.3
 Negotiated protocol          TLSv1.3
 Negotiated cipher            TLS_AES_256_GCM_SHA384, 253 bit ECDH (X25519)
 Cipher order
    TLSv1.2:   ECDHE-RSA-AES128-GCM-SHA256 ECDHE-RSA-AES256-GCM-SHA384
ECDHE-RSA-CHACHA20-POLY1305 ECDHE-RSA-AES128-SHA256 ECDHE-RSA-AES256-SHA384
               ECDHE-RSA-AES128-SHA ECDHE-RSA-AES256-SHA AES128-GCM-SHA256
AES128-SHA AES256-SHA
```

```
<output omitted for brevity>
 Server extended key usage    TLS Web Server Authentication, TLS Web Client
Authentication
 Serial                       03C3A18FC6D788539C0BFEE16E585A4F6436 (OK: length 18)
 Fingerprints                 SHA1 4AB6BA78E972C99CA947A501CFEA83AA4D1D3323
                              SHA256
489964F5A7ABB9F8ADC0082652A6F978E379F2496247045850C96ACD809FFE32
 Common Name (CN)             websploit.org  (CN in response to request w/o
SNI: *.github.io )
 subjectAltName (SAN)         websploit.org
 Issuer                       R3 (Let's Encrypt from US)
 Trust (hostname)             Ok via SAN (SNI mandatory)
 Chain of trust               Ok
 EV cert (experimental)       no
 ETS/"eTLS", visibility info  not present
 Certificate Validity (UTC)   55 >= 30 days (2023-08-15 22:14 --> 2023-11-13 22:14)
 # of certificates provided   3
 Certificate Revocation List  --
 OCSP URI                     http://r3.o.lencr.org
 OCSP stapling                offered, not revoked
 OCSP must staple extension   --
 DNS CAA RR (experimental)    not offered
 Certificate Transparency     yes (certificate extension)
 <output omitted for brevity>
 Testing vulnerabilities
 Heartbleed (CVE-2014-0160)                not vulnerable (OK), no heartbeat extensi
 CCS (CVE-2014-0224)                       not vulnerable (OK)
 Ticketbleed (CVE-2016-9244), experiment.  not vulnerable (OK)
 ROBOT                                     not vulnerable (OK)
 Secure Renegotiation (RFC 5746)           supported (OK)
 Secure Client-Initiated Renegotiation     likely not vulnerable (OK), timed out
 CRIME, TLS (CVE-2012-4929)                not vulnerable (OK)
 BREACH (CVE-2013-3587)                    potentially NOT ok, "gzip" HTTP
 compression detected. - only supplied "/" tested
                                           Can be ignored for static pages or if no
secrets in the page
 POODLE, SSL (CVE-2014-3566)               not vulnerable (OK), no SSLv3 support
 TLS_FALLBACK_SCSV (RFC 7507)              No fallback possible (OK), no protocol
below TLS 1.2 offered
 SWEET32 (CVE-2016-2183, CVE-2016-6329)    not vulnerable (OK)
 FREAK (CVE-2015-0204)                     not vulnerable (OK)
 DROWN (CVE-2016-0800, CVE-2016-0703)      not vulnerable on this host and port (OK)
                                           make sure you don't use this certificate
elsewhere with SSLv2 enabled services

https://search.censys.io/search?resource=hosts&virtual_hosts=INCLUDE&q=489964F5A7AB
B9F8ADC0082652A6F978E379F2496247045850C96ACD809FFE32
 LOGJAM (CVE-2015-4000), experimental      not vulnerable (OK): no DH EXPORT
```

```
ciphers, no DH key detected with <= TLS 1.2
 BEAST (CVE-2011-3389)                    not vulnerable (OK), no SSL3 or TLS1
 LUCKY13 (CVE-2013-0169), experimental    potentially VULNERABLE, uses cipher
block chaining (CBC) ciphers with TLS. Check patches
 RC4 (CVE-2013-2566, CVE-2015-2808)       no RC4 ciphers detected (OK)

 Testing 370 ciphers via OpenSSL plus sockets against the server, ordered by
encryption strength
<output omitted for brevity>
```

The highlighted lines in **Example 9-12** show how the **testssl** tool is able to test for weak crypto algorithms and for known vulnerabilities.

Obtaining Sensitive Information About External and Internal Hosts Using Certificate Transparency

You can also leverage certificate transparency to reveal additional information and enumerate subdomains. What is certificate transparency? More than a decade ago, there was a major attack against DigiNotar (an organization that creates, maintains, and authorizes digital certificates for many companies and government institutions). This attack (along with other similar attacks) caused concerns around how organizations generate and manage digital certificates and, subsequently, why certificate transparency was created to better detect the issuing of malicious certificates. The goal of certificate transparency is for any organization or individual to be able to "transparently" verify the issuance of a digital certificate.

Certificate transparency allows certificate authorities (CAs) to provide details about all related certificates that have been issued for a given domain and organization. Attackers can also use this information to reveal what other subdomains and systems an organization may own.

Tip

You can obtain detailed information about certificate transparency at **https://certificate.transparency.dev**.

Websites such as **https://crt.sh** allow you to obtain detailed certificate transparency information about any given domain. I created a tool called CertSPY that leverages certificate transparency logs accessible through the crt.sh site. You can access the tool source code at **https://github.com/santosomar/certspy**.

You can install CertSPY by using the **pip3 install certspy** command. **Example 9-13** demonstrates how to use the tool to obtain certificate information for the **secretcorp.org** domain.

Example 9-13 Using CertSPY to Obtain Certificate Transparency Information

Click here to view code image

```
┌──(omar㉿websploit)-[~]
└─$ certspy secretcorp.org
[
    {
        "issuer_ca_id": 183267,
        "issuer_name": "C=US, O=Let's Encrypt, CN=R3",
        "common_name": "secretcorp.org",
        "name_value": "secretcorp.org",
        "id": 10254588889,
        "entry_timestamp": "2023-08-30T08:49:46.284",
        "not_before": "2023-08-30T07:49:06",
        "not_after": "2023-11-28T07:49:05",
        "serial_number": "046cb5a18ef41e26f9867cfdb61d28452047"
    },
    {
        "issuer_ca_id": 183267,
        "issuer_name": "C=US, O=Let's Encrypt, CN=R3",
        "common_name": "mail.secretcorp.org",
        "name_value": "mail.secretcorp.org",
        "id": 10039294064,
        "entry_timestamp": "2023-08-01T04:19:56.363",
        "not_before": "2023-08-01T03:19:55",
        "not_after": "2023-10-30T03:19:54",
        "serial_number": "049b42b3d9dfad81b882209188f2dd3416e4"
    },
    {
        "issuer_ca_id": 183267,
        "issuer_name": "C=US, O=Let's Encrypt, CN=R3",
        "common_name": "app1.secretcorp.org",
        "name_value": "app1.secretcorp.org",
        "id": 10038384450,
        "entry_timestamp": "2023-08-01T01:00:56.816",
        "not_before": "2023-08-01T00:00:56",
        "not_after": "2023-10-30T00:00:55",
        "serial_number": "046bbf0c4112b9c2a1a8b30d8e50c8050264"
    },

<output omitted for brevity>
```

Example 9-14 shows how to obtain the common_name (that is, host-names) using grep.

Example 9-14 Obtaining the Hostnames Only

Click here to view code image

```
┌──(omar㉿websploit)-[~]
└─$ certspy secretcorp.org | grep common_name
        "common_name": "secretcorp.org",
        "common_name": "secretcorp.org",
        "common_name": "mail.secretcorp.org",
        "common_name": "mail.secretcorp.org",
        "common_name": "app1.secretcorp.org",
        "common_name": "app1.secretcorp.org",
        "common_name": "internal.secretcorp.org",
        "common_name": "internal.secretcorp.org",
        "common_name": "cloud.secretcorp.org",
        "common_name": "cloud.secretcorp.org",
        "common_name": "finance-app.secretcorp.org",
        "common_name": "finance-app.secretcorp.org",
        "common_name": "backdoor.secretcorp.org",
        "common_name": "backdoor.secretcorp.org",
        "common_name": "vpn.secretcorp.org",
        "common_name": "vpn.secretcorp.org",
        "common_name": "secretcorp.org",
        "common_name": "secretcorp.org",
```

Tip

You can learn more about CertSPY at https://becomingahacker.org/introducing-certspy-89da08beca25.

Leveraging Password Dumps

Attackers can leverage password dumps from previous breaches. There are a number of ways that attackers can get access to such password dumps. These sources include Pastebin, dark web sites, and even GitHub in some cases. Different tools and websites make this task very easy. An example of a tool that allows you to find email addresses and passwords exposed in previous breaches is h8mail. You can install h8mail using the **pip3 install h8mail** command, as demonstrated in Example 9-15. This example also shows the h8mail command-line usage.

Example 9-15 Installing and Using h8mail

Click here to view code image

```
root@websploit# pip3 install h8mail
Collecting h8mail
  Downloading h8mail-2.5.5-py3-none-any.whl (33 kB)
Requirement already satisfied: requests in /usr/lib/python3/dist-packages
(from h8mail) (2.23.0)
Installing collected packages: h8mail
Successfully installed h8mail-2.5.5
root@websploit# h8mail -h
usage: h8mail [-h] [-t USER_TARGETS [USER_TARGETS ...]]
              [-u USER_URLS [USER_URLS ...]] [-q USER_QUERY] [--loose]
              [-c CONFIG_FILE [CONFIG_FILE ...]] [-o OUTPUT_FILE]
              [-j OUTPUT_JSON] [-bc BC_PATH] [-sk]
              [-k CLI_APIKEYS [CLI_APIKEYS ...]]
              [-lb LOCAL_BREACH_SRC [LOCAL_BREACH_SRC ...]]
              [-gz LOCAL_GZIP_SRC [LOCAL_GZIP_SRC ...]] [-sf]
              [-ch [CHASE_LIMIT]] [--power-chase] [--hide] [--debug]
              [--gen-config]

Email information and password lookup tool
optional arguments:
  -h, --help            show this help message and exit
  -t USER_TARGETS [USER_TARGETS ...], --targets USER_TARGETS [USER_TARGETS ...]
                        Either string inputs or files. Supports email pattern
                        matching from input or file, filepath globing and
                        multiple arguments
  -u USER_URLS [USER_URLS ...], --url USER_URLS [USER_URLS ...]
                        Either string inputs or files. Supports URL pattern
                        matching from input or file, filepath globing and
                        multiple arguments. Parse URLs page for emails.
                        Requires http:// or https:// in URL.
  -q USER_QUERY, --custom-query USER_QUERY
                        Perform a custom query. Supports username, password,
                        ip, hash, domain. Performs an implicit "loose" search
                        when searching locally
  --loose               Allow loose search by disabling email pattern
                        recognition. Use spaces as pattern separators
  -c CONFIG_FILE [CONFIG_FILE ...], --config CONFIG_FILE [CONFIG_FILE ...]
                        Configuration file for API keys. Accepts keys from
                        Snusbase, WeLeakInfo, Leak-Lookup, HaveIBeenPwned,
                        Emailrep, Dehashed and hunterio
  -o OUTPUT_FILE, --output OUTPUT_FILE
                        File to write CSV output
  -j OUTPUT_JSON, --json OUTPUT_JSON
                        File to write JSON output
```

```
  -bc BC_PATH, --breachcomp BC_PATH
                        Path to the breachcompilation torrent folder. Uses the
                        query.sh script included in the torrent
  -sk, --skip-defaults  Skips Scylla and HunterIO check. Ideal for local scans
  -k CLI_APIKEYS [CLI_APIKEYS ...], --apikey CLI_APIKEYS [CLI_APIKEYS ...]
                        Pass config options. Supported format: "K=V,K=V"
  -lb LOCAL_BREACH_SRC [LOCAL_BREACH_SRC ...], --local-breach LOCAL_BREACH_SRC
[LOCAL_BREACH_SRC ...]
                        Local cleartext breaches to scan for targets. Uses
                        multiprocesses, one separate process per file, on
                        separate worker pool by arguments. Supports file or
                        folder as input, and filepath globing
  -gz LOCAL_GZIP_SRC [LOCAL_GZIP_SRC ...], --gzip LOCAL_GZIP_SRC
[LOCAL_GZIP_SRC ...]
                        Local tar.gz (gzip) compressed breaches to scans for
                        targets. Uses multiprocesses, one separate process per
                        file. Supports file or folder as input, and filepath
                        globing. Looks for 'gz' in filename
  -sf, --single-file    If breach contains big cleartext or tar.gz files, set
                        this flag to view the progress bar. Disables
                        concurrent file searching for stability
  -ch [CHASE_LIMIT], --chase [CHASE_LIMIT]
                        Add related emails from hunter.io to ongoing target
                        list. Define number of emails per target to chase.
                        Requires hunter.io private API key if used without
                        power-chase
  --power-chase         Add related emails from ALL API services to ongoing
                        target list. Use with --chase
  --hide                Only shows the first 4 characters of found passwords
                        to output. Ideal for demonstrations
  --debug               Print request debug information
  --gen-config, -g      Generates a configuration file template in the current
                        working directory & exits. Will overwrite existing
                        h8mail_config.ini file
```

The following tools also allow you to search for breach data dumps:

- **WhatBreach:** **https://github.com/Ekultek/WhatBreach**
- **BaseQuery:** **https://github.com/jayyogesh/BaseQuery**
- **Buster:** **https://github.com/sham00n/buster**
- **Scavenger:** **https://github.com/rndinfosecguy/Scavenger**

Tools like h8mail and WhatBreach take advantage of breached data repositories of websites such as **haveibeenpwned.com**, **weleakinfo.com**, and **snusbase.com**.

Recon from File Metadata

You can obtain a lot of information from metadata in files such as images; Microsoft Word, Excel, PowerPoint documents; and more. For instance, the Exchangeable Image File Format (Exif) is a specification that defines the formats for images, sound, and supplementary tags used by digital cameras, mobile phones, scanners, and other systems that process image and sound files.

Several tools can show the Exif details. One of the most popular tools is the Exif tool demonstrated in **Example 9-16**. It shows the Exif data of a digital image (picture) captured by an iPhone.

Example 9-16 Using the Exif Tool

Click here to view code image

```
┌─[omar@websploit]─[~]
└── $exiftool IMG_4730.jpg
ExifTool Version Number         : 12.06
File Name                       : IMG_4730.jpg
Directory                       : .
File Size                       : 2.4 MB
File Modification Date/Time     : 2024:06:20 21:33:36-04:00
File Access Date/Time           : 2024:06:20 21:33:36-04:00
File Inode Change Date/Time     : 2024:06:20 21:33:36-04:00
File Permissions                : rw-r--r--
File Type                       : JPEG
File Type Extension             : jpg
MIME Type                       : image/jpeg
JFIF Version                    : 1.01
Exif Byte Order                 : Big-endian (Motorola, MM)
Make                            : Apple
Camera Model Name               : iPhone 14 Pro Max
Orientation                     : Horizontal (normal)
X Resolution                    : 72
Y Resolution                    : 72
Resolution Unit                 : inches
Software                        : 14.6
Modify Date                     : 2024:06:20 17:45:44
Host Computer                   : iPhone 15 Pro Max
Tile Width                      : 512
Tile Length                     : 512
Exposure Time                   : 1/887
F Number                        : 1.6
Exposure Program                : Program AE
ISO                             : 32
```

```
Exif Version                    : 0232
Date/Time Original              : 2024:06:20 17:45:44
Create Date                     : 2024:06:20 17:45:44
Offset Time                     : -04:00
Offset Time Original            : -04:00
Offset Time Digitized           : -04:00
Components Configuration        : Y, Cb, Cr, -
Shutter Speed Value             : 1/887
Aperture Value                  : 1.6
Brightness Value                : 7.700648484
Exposure Compensation           : 0
Metering Mode                   : Multi-segment
<output omitted for brevity>
Composite Image                 : General Composite Image
GPS Version ID                  : 2.2.0.0
GPS Latitude Ref                : North
GPS Longitude Ref               : West
GPS Altitude Ref                : Above Sea Level
GPS Speed Ref                   : km/h
GPS Speed                       : 0.2300000042
GPS Img Direction Ref           : True North
GPS Img Direction               : 74.98474114
GPS Dest Bearing Ref            : True North
GPS Dest Bearing                : 74.98474114
GPS Horizontal Positioning Error: 15.67681275 m
Compression                     : JPEG (old-style)
Thumbnail Offset                : 2650
Thumbnail Length                : 6523
XMP Toolkit                     : XMP Core 5.5.0
Creator Tool                    : 14.6
Date Created                    : 2024:06:20 17:45:44
<output omitted for brevity>
GPS Altitude                    : 112.7 m Above Sea Level
GPS Latitude                    : 35 deg 46' 78.6382" N
GPS Longitude                   : 78 deg 38' 30.4793" W
Circle Of Confusion             : 0.006 mm
Field Of View                   : 69.4 deg
Focal Length                    : 5.1 mm (35 mm equivalent: 26.0 mm)
GPS Position                    : 35 deg 46' 49.69" N, 78 deg 38' 30.4793" W
Hyperfocal Distance             : 2.76 m
Light Value                     : 12.8
```

The highlighted lines in **Example 9-16** show the GPS data of the location where the image was taken.

Strategic Search Engine Analysis/Enumeration

Most of us use search engines such as DuckDuckGo, Bing, Google, and Perplexity to locate information. What you might not know is that search engines, such as Google, can perform much more powerful searches than most people ever dream of. Not only can Google translate documents, perform news searches, and do image searches, but it can also be used by hackers and attackers to do something that has been termed *Google hacking*.

When basic search techniques are combined with advanced operators, Google can become a powerful vulnerability search tool. The following are some advanced operators:

- **Filetype**: Directs Google to search only within the text of a particular type of file. Example: filetype:xls
- **Inurl**: Directs Google to search only within the specified URL of a document. Example: inurl:search-text
- **Link**: Directs Google to search within hyperlinks for a specific term. Example: link:**www.domain.com**
- **Intitle**: Directs Google to search for a term within the title of a document. Example: intitle: “Index of.etc”

By using these advanced operators in combination with key terms, you can use Google to uncover many pieces of sensitive information that shouldn’t be revealed. A term even exists for the people who blindly post this information on the Internet; they are called *Google dork*s. To see how this works, enter the following phrase into Google:

intext:JSESSIONID OR intext:PHPSESSID inurl:access.log ext:log

This query searches in a URL for the session IDs that could be used to potentially impersonate users. In our case, the search found more than 100 sites that store sensitive session IDs in publicly accessible logs. If these IDs have not timed out, they could be used to gain access to restricted resources.

You can use advanced operators to search for many types of data. The following is another example of a Google search string (or Google dork) that can reveal passwords of web applications:

"public $user =" | "public $password = " | "public $secret =" | "public $db =" ext:txt | ext:log -git

Now that we have discussed some basic Google search techniques, let's look at advanced Google hacking. If you have never visited the Google Hacking Database (GHDB) repositories, I suggest that you visit **https://www.exploit-db.com/google-hacking-database/**. GHDB has the following search categories:

- Footholds
- Files containing usernames
- Sensitive directories
- Web server detection
- Vulnerable files
- Vulnerable servers
- Error messages
- Files containing juicy info
- Files containing passwords
- Sensitive online shopping info
- Network or vulnerability data
- Pages containing login portals
- Different online devices
- Advisories and vulnerabilities

GHDB is a community effort. Anyone can upload a new Google dork to perform these types of searches. Once you start playing with the dorks in GHDB, you will be surprised by the unbelievable things found by Google hacking. GHDB has made using Google dorks easier, but it's not your only option. Later you will learn about additional tools that can be used to perform similar searches (such as Recon-ng).

Website Archive/Caching

Several organizations archive and cache website data on the Internet. One of the most popular repositories is the Wayback Machine of the Internet Archive (**https://archive.org/web**).

Let's go back in time and see how Cisco's website looked on November 3, 1999 (shown in **Figure 9-2**). You can access the archive of the site shown in **Figure 9-2** by navigating to **https://web.archive.org/web/19991103121048/http://www.cisco.com**.

Figure 9-2

The Internet Archive Wayback Machine

Public Source-Code Repositories, Secrets, and Other Sensitive Information

An attacker can obtain extremely valuable information from public source-code repositories such as GitHub and GitLab. Most of the applications and products we consume today use open-source software. The source code can be obtained in public repositories. Attackers can find vulnerabilities in those software packages and use them to their advantage. Similarly, as a penetration tester, you can obtain valuable information from these public repositories. Even if you do not immediately find security vulnerabilities in the code, these repositories can give you insights into the architecture and underlying code used in the organization's applications and infrastructure.

TruffleHog is a good tool used primarily for digging deep into the repositories and other data sources to find secrets that might be buried there. You can obtain the tool from **https://github.com/trufflesecurity/trufflehog**.

You can install TruffleHog easily by cloning the GitHub repository and using the **go install** command, as shown in **Example 9-17**.

Example 9-17 Installing TruffleHog

Click here to view code image

```
git clone https://github.com/trufflesecurity/trufflehog.git
cd trufflehog; go install
```

TruffleHog supports different arguments, each designed to scan a specific source of data.

- **git**: Used to scan local or remote Git repositories. It sifts through the commits to find potentially sensitive information that has been committed.
- **github**: Used to scan GitHub repositories for sensitive data. It can be used to scan both public and private repositories, provided you have the necessary access rights to the private repositories.
- **gitlab**: Similar to the **github** option but designed to work with GitLab repositories.
- **docker**: Used to scan Docker images and containers. It helps in identifying any sensitive information that might be embedded in Docker images.
- **S3**: Used to scan Amazon S3 buckets. It can help you identify any sensitive files or data that might be stored in an S3 bucket.
- **circleci**: Used to scan CircleCI configurations and build files for secrets or sensitive information.

Performing Reconnaissance with Recon-ng

So far, you have learned a number of individual sources and tools used for information gathering. These tools are all very effective for their specific uses; however, wouldn't it be great if there were a tool that could pull together all these different functions? This is where Recon-ng comes in. It is a framework originally developed by Tim Tomes, and it is now supported by Black Hills Information Security (a cybersecurity company). This tool was developed in Python with Metasploit **msfconsole** in mind. If you've used the Metasploit console before, Recon-ng will be familiar and easy to understand.

Recon-ng has a modular framework, which makes it easy to develop and integrate new functionality. It is highly effective in social networking site enumeration because of its use of APIs to gather information. It also includes a reporting feature that allows you to export using different report formats. Because you will always need to provide some kind of deliverable in any testing you do, Recon-ng is especially valuable.

To start using Recon-ng, you simply run **recon-ng** from a new terminal window. **Example 9-18** shows the command and the initial menu that Recon-ng starts with.

Example 9-18 Starting Recon-ng

Click here to view code image

```
┌─[omar@websploit]─[~]
└── $recon-ng
[*] Version check disabled.

                                        /\
                                       / \\ /\
    Sponsored by...                 /\  /\/  \\V  \/\
                                   / \\/ // \\\\\ \\ \/\
                                  // // BLACK HILLS \/ \\
                                 www.blackhillsinfosec.com

                  ____   ____   ____   ____ _____ _  ____   ____  ____
                 |____] | ___/ |____| |       |   | |____  |____ |
                 |      |   \_ |    | |____   |   |  ____| |____ |____
                                    www.practisec.com

                       [recon-ng v5.1.2, Tim Tomes (@lanmaster53)]

[4] Recon modules
[1] Discovery modules

[recon-ng][default] >
```

From the initial screen, you can see the current number of modules that are installed in Recon-ng. To get an idea of what commands are available at the Recon-ng command-line tool, you can simply type **help** and press **Enter**. **Example 9-19** shows the output of the **help** command.

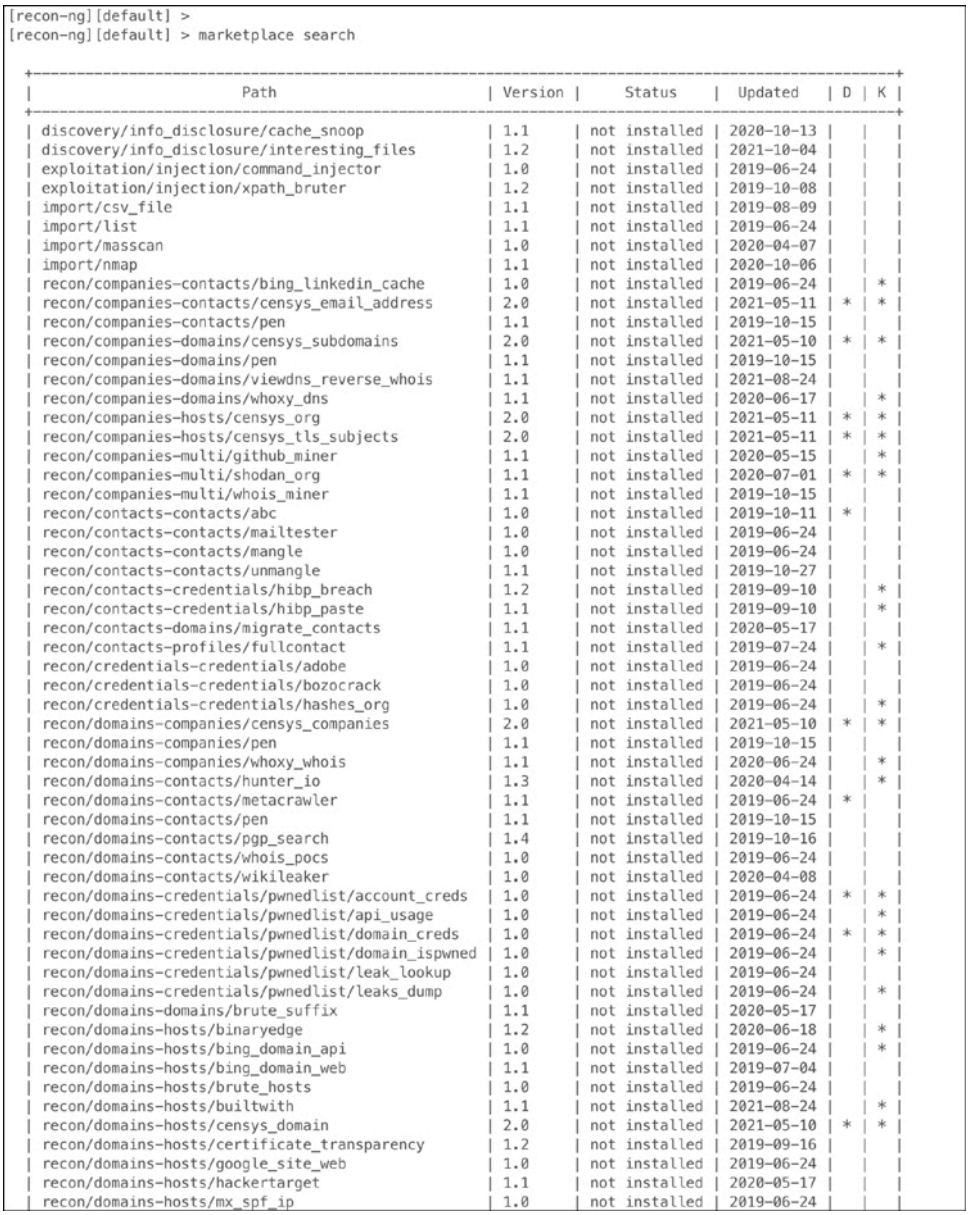
Example 9-19 Starting Recon-ng

Click here to view code image

```
[recon-ng][default] > help
Commands (type [help|?] <topic>):
---------------------------------
back            Exits the current context
dashboard       Displays a summary of activity
db              Interfaces with the workspace's database
```

```
exit            Exits the framework
help            Displays this menu
index           Creates a module index (dev only)
keys            Manages third party resource credentials
marketplace     Interfaces with the module marketplace
modules         Interfaces with installed modules
options         Manages the current context options
pdb             Starts a Python Debugger session (dev only)
script          Records and executes command scripts
shell           Executes shell commands
show            Shows different framework items
snapshots       Manages workspace snapshots
spool           Spools output to a file
workspaces      Manages workspaces
[recon-ng][default] >
```

Before you can start gathering information using the Recon-ng tool, you need to understand what modules are available. Recon-ng comes with a "marketplace" where you can search for available modules to be installed. You can use the **marketplace search** command to search for all the available modules in Recon-ng, as demonstrated in **Figure 9-3**.



```
[recon-ng][default] >
[recon-ng][default] > marketplace search

+-------------------------------------------------------------------------------------------+
|                                Path                                | Version |     Status    |  Updated   | D | K |
+-------------------------------------------------------------------------------------------+
| discovery/info_disclosure/cache_snoop                              | 1.1     | not installed | 2020-10-13 |   |   |
| discovery/info_disclosure/interesting_files                        | 1.2     | not installed | 2021-10-04 |   |   |
| exploitation/injection/command_injector                            | 1.0     | not installed | 2019-06-24 |   |   |
| exploitation/injection/xpath_bruter                                | 1.2     | not installed | 2019-10-08 |   |   |
| import/csv_file                                                    | 1.1     | not installed | 2019-08-09 |   |   |
| import/list                                                        | 1.1     | not installed | 2019-06-24 |   |   |
| import/masscan                                                     | 1.0     | not installed | 2020-04-07 |   |   |
| import/nmap                                                        | 1.1     | not installed | 2020-10-06 |   |   |
| recon/companies-contacts/bing_linkedin_cache                       | 1.0     | not installed | 2019-06-24 |   | * |
| recon/companies-contacts/censys_email_address                      | 2.0     | not installed | 2021-05-11 | * | * |
| recon/companies-contacts/pen                                       | 1.1     | not installed | 2019-10-15 |   |   |
| recon/companies-domains/censys_subdomains                          | 2.0     | not installed | 2021-05-10 | * | * |
| recon/companies-domains/pen                                        | 1.1     | not installed | 2019-10-15 |   |   |
| recon/companies-domains/viewdns_reverse_whois                      | 1.1     | not installed | 2021-08-24 |   |   |
| recon/companies-domains/whoxy_dns                                  | 1.1     | not installed | 2020-06-17 |   | * |
| recon/companies-hosts/censys_org                                   | 2.0     | not installed | 2021-05-11 | * | * |
| recon/companies-hosts/censys_tls_subjects                          | 2.0     | not installed | 2021-05-11 | * | * |
| recon/companies-multi/github_miner                                 | 1.1     | not installed | 2020-05-15 |   | * |
| recon/companies-multi/shodan_org                                   | 1.1     | not installed | 2020-07-01 | * | * |
| recon/companies-multi/whois_miner                                  | 1.1     | not installed | 2019-10-15 |   |   |
| recon/contacts-contacts/abc                                        | 1.0     | not installed | 2019-10-11 | * |   |
| recon/contacts-contacts/mailtester                                 | 1.0     | not installed | 2019-06-24 |   |   |
| recon/contacts-contacts/mangle                                     | 1.0     | not installed | 2019-06-24 |   |   |
| recon/contacts-contacts/unmangle                                   | 1.1     | not installed | 2019-10-27 |   |   |
| recon/contacts-credentials/hibp_breach                             | 1.2     | not installed | 2019-09-10 |   | * |
| recon/contacts-credentials/hibp_paste                              | 1.1     | not installed | 2019-09-10 |   | * |
| recon/contacts-domains/migrate_contacts                            | 1.1     | not installed | 2020-05-17 |   |   |
| recon/contacts-profiles/fullcontact                                | 1.1     | not installed | 2019-07-24 |   | * |
| recon/credentials-credentials/adobe                                | 1.0     | not installed | 2019-06-24 |   |   |
| recon/credentials-credentials/bozocrack                            | 1.0     | not installed | 2019-06-24 |   |   |
| recon/credentials-credentials/hashes_org                           | 1.0     | not installed | 2019-06-24 |   | * |
| recon/domains-companies/censys_companies                           | 2.0     | not installed | 2021-05-10 | * | * |
| recon/domains-companies/pen                                        | 1.1     | not installed | 2019-10-15 |   |   |
| recon/domains-companies/whoxy_whois                                | 1.1     | not installed | 2020-06-24 |   | * |
| recon/domains-contacts/hunter_io                                   | 1.3     | not installed | 2020-04-14 |   | * |
| recon/domains-contacts/metacrawler                                 | 1.1     | not installed | 2019-06-24 | * |   |
| recon/domains-contacts/pen                                         | 1.1     | not installed | 2019-10-15 |   |   |
| recon/domains-contacts/pgp_search                                  | 1.4     | not installed | 2019-10-16 |   |   |
| recon/domains-contacts/whois_pocs                                  | 1.0     | not installed | 2019-06-24 |   |   |
| recon/domains-contacts/wikileaker                                  | 1.0     | not installed | 2020-04-08 |   |   |
| recon/domains-credentials/pwnedlist/account_creds                  | 1.0     | not installed | 2019-06-24 | * | * |
| recon/domains-credentials/pwnedlist/api_usage                      | 1.0     | not installed | 2019-06-24 |   | * |
| recon/domains-credentials/pwnedlist/domain_creds                   | 1.0     | not installed | 2019-06-24 | * | * |
| recon/domains-credentials/pwnedlist/domain_ispwned                 | 1.0     | not installed | 2019-06-24 |   | * |
| recon/domains-credentials/pwnedlist/leak_lookup                    | 1.0     | not installed | 2019-06-24 |   |   |
| recon/domains-credentials/pwnedlist/leaks_dump                     | 1.0     | not installed | 2019-06-24 |   | * |
| recon/domains-domains/brute_suffix                                 | 1.1     | not installed | 2020-05-17 |   |   |
| recon/domains-hosts/binaryedge                                     | 1.2     | not installed | 2020-06-18 |   | * |
| recon/domains-hosts/bing_domain_api                                | 1.0     | not installed | 2019-06-24 |   | * |
| recon/domains-hosts/bing_domain_web                                | 1.1     | not installed | 2019-07-04 |   |   |
| recon/domains-hosts/brute_hosts                                    | 1.0     | not installed | 2019-06-24 |   |   |
| recon/domains-hosts/builtwith                                      | 1.1     | not installed | 2021-08-24 |   | * |
| recon/domains-hosts/censys_domain                                  | 2.0     | not installed | 2021-05-10 | * | * |
| recon/domains-hosts/certificate_transparency                       | 1.2     | not installed | 2019-09-16 |   |   |
| recon/domains-hosts/google_site_web                                | 1.0     | not installed | 2019-06-24 |   |   |
| recon/domains-hosts/hackertarget                                   | 1.1     | not installed | 2020-05-17 |   |   |
| recon/domains-hosts/mx_spf_ip                                      | 1.0     | not installed | 2019-06-24 |   |   |
```

Figure 9-3

The Recon-ng Marketplace Search

Figure 9-3 shows only the first few modules available in Recon-ng. The letter *D* in the table header indicates that the module has dependencies. The letter *K* indicates that you need an API key to be able to use the resource used in such module. For example, the module with the path recon/companies-contacts/censys_email_address has dependencies and needs an API key to be able to query the Censys database. Censys is a popular resource to query OSINT data.

You can refresh the data about the available modules by using the **marketplace refresh** command, as shown in Example 9-20.

Example 9-20 Refreshing the Recon-ng Marketplace Data

Click here to view code image

```
[recon-ng][default] > marketplace refresh
[*] Marketplace index refreshed.
```

Let's perform a quick search to find different subdomains of one of my domains (h4cker.org). For this example, let's use the module bing_domain_web to try to find any subdomains leveraging the Bing search engine. You can perform a keyword search for any modules by using the **marketplace search <keyword>** command, as demonstrated in **Example 9-21**.

Example 9-21 Marketplace Keyword Search

Click here to view code image

```
[recon-ng][default] > marketplace search bing
[*] Searching module index for 'bing'...
+-------------------------------------------------------------------------+
|          Path            | Version |     Status    |  Updated   | D | K |
+-------------------------------------------------------------------------+
| recon/companies-contacts/bing_linkedin_cache   | 1.0     | not installed |
2019-06-24 |   | * |
| recon/domains-hosts/bing_domain_api            | 1.0     | not installed |
2019-06-24 |   | * |
| recon/domains-hosts/bing_domain_web            | 1.1     | not installed |
2019-07-04 |   |   |
| recon/hosts-hosts/bing_ip                      | 1.0     | not installed |
2019-06-24 |   | * |
| recon/profiles-contacts/bing_linkedin_contacts | 1.1     | not installed |
2019-10-08 |   | * |
+-------------------------------------------------------------------------+

  D = Has dependencies. See info for details.
  K = Requires keys. See info for details.
```

Several results matched the bing keyword. However, the one that we are interested in is recon/domains-hosts/bing_domain_web. You can install the module by using the **marketplace install** command, as shown in **Example 9-22**.

Example 9-22 Installing a Recon-ng Module

Click here to view code image

```
[recon-ng][default] > marketplace install recon/domains-hosts/bing_domain_web
[*] Module installed: recon/domains-hosts/bing_domain_web
```

```
[*] Reloading modules...
[recon-ng][default] >
```

You can use the **modules search** command, as shown in **Example 9-23**, to show all the modules that have been installed in Recon-ng.

Example 9-23 Recon-ng Installed Modules

Click here to view code image

```
[recon-ng][default] > modules search
  Discovery
  ---------
    discovery/info_disclosure/interesting_files
  Recon
  -----
    recon/domains-hosts/bing_domain_web
    recon/domains-hosts/brute_hosts
    recon/domains-hosts/certificate_transparency
    recon/domains-hosts/netcraft
[recon-ng][default] >
```

To load the module that you would like to use, use the **modules load** command, as shown in **Example 9-24**. In this example, the bing_domain_web module is loaded. Notice that the prompt changed, including the name of the loaded module. After the module is loaded, you can display the module options using the **info** command (also demonstrated in **Example 9-24**).

Example 9-24 Loading an Installed Module in Recon-ng

Click here to view code image

```
[recon-ng][default] > modules load recon/domains-hosts/bing_domain_web
[recon-ng][default][bing_domain_web] > info
    Name: Bing Hostname Enumerator
    Author: Tim Tomes (@lanmaster53)
   Version: 1.1
Description:
  Harvests hosts from Bing.com by using the 'site' search operator. Updates the
'hosts' table with the results.

Options:
  Name    Current Value  Required  Description
  ------  -------------  --------  -----------
```

```
  SOURCE  example.com    yes       source of input (see 'info' for details)

Source Options:
  default        SELECT DISTINCT domain FROM domains WHERE domain IS NOT NULL
  <string>       string representing a single input
  <path>         path to a file containing a list of inputs
  query <sql>    database query returning one column of inputs
[recon-ng][default][bing_domain_web] >
```

Now, let's change the source (the domain to be used to find its subdomains) using the **options set SOURCE** command, as demonstrated in **Example 9-25**. After the source domain is set, type **run** to run the query (as also shown in **Example 9-25**).

Example 9-25 Setting the Source Domain and Running the Query

Click here to view code image

```
[[recon-ng][default][bing_domain_web] > options set SOURCE h4cker.org
SOURCE => h4cker.org
[recon-ng][default][bing_domain_web] > run
----------
H4CKER.ORG
----------
[*] URL: https://www.bing.com/search?first=0&q=domain%3Ah4cker.org
[*] Country: None
[*] Host: bootcamp.h4cker.org
[*] Ip_Address: None
[*] Latitude: None
[*] Longitude: None
[*] Notes: None
[*] Region: None
[*] -------------------------------------------------
[*] Country: None
[*] Host: webapps.h4cker.org
[*] Ip_Address: None
<output omitted for brevity>
[*] -------------------------------------------------
[*] Country: None
[*] Host: lpb.h4cker.org
<output omitted for brevity>
[*] -------------------------------------------------
[*] Country: None
[*] Host: malicious.h4cker.org
<output omitted for brevity>
SUMMARY
-------
```

```
[*] 4 total (0 new) hosts found.
[recon-ng][default][bing_domain_web] >
```

Four subdomains (shown in the highlighted lines in **Example 9-25**) were found using the bing_domain_web module.

Tip

You should also combine other modules to find additional information about a specific target. For example, you can use the recon/domains-hosts/brute_hosts module to use wordlists and perform DNS queries to find additional subdomains. Hacking is all about the process of thinking like an attacker and developing a good methodology. I strongly recommend that you "go beyond the tool" and default behavior and develop your own methodology by combining tools and other resources to find information and potential vulnerabilities to exploit.

What makes Recon-ng so powerful is the way it is able to use the application programming interfaces (APIs) of different OSINT resources to gather information. Modules included are able to query sites such as Facebook, Indeed, Instagram, LinkedIn, and YouTube. You can also brute-force DNS subdomains, gather information on certificate transparency, and query many other OSINT tools like Shodan. So, what is that thing called Shodan?

Shodan

Shodan is the name of an organization that is scanning the Internet 24 hours a day, 365 days a year. The results of those scans are stored in a database that you can query by going to shodan.io or by using an API. You can use Shodan to query for vulnerable hosts, IoT devices, and many other systems that should not be exposed or connected to the public Internet.

Figure 9-4 shows different categories of systems found by Shodan scans, including industrial control systems, databases, network infrastructure devices, video games, and more.

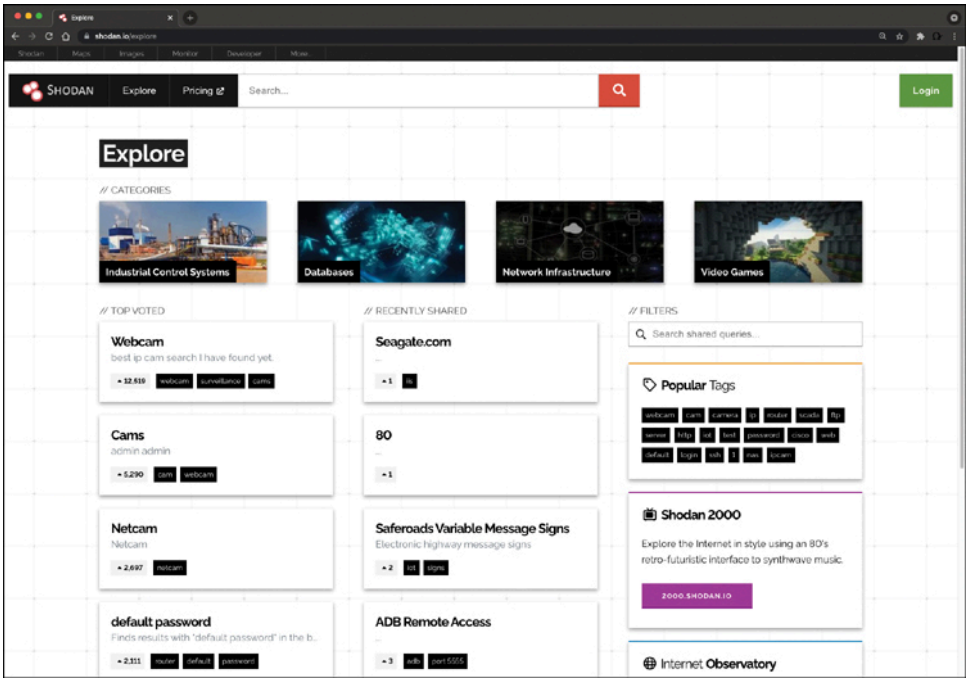


Figure 9-4

Exploring the Shodan Database

Note

Shodan offers two primary lifetime subscription options. A one-time payment grants users lifetime access to advanced features on Shodan, such as unlimited search results, API access, and access to advanced filters. Shodan also offers a free upgrade to students, professors, or IT staff at universities.

Example 9-26 shows a query performed using the Shodan API to find network infrastructure devices that are running a broken protocol called the "Cisco Smart Install" protocol. Attackers have leveraged that protocol for years to compromise different infrastructures. Cisco removed this protocol from its systems many years ago. However, many people are still using it in devices connected to the public Internet. Cisco has warned customers for many years in a security advisory that can be accessed at **https://tools.cisco.com/security/center/content/CiscoSecurityAdvisory/cisco-sa-20170214-smi**.

Example 9-26 Setting the Source Domain and Running the Query

Click here to view code image

```
┌──(omar㉿websploit)-[~]
└─$ shodan search "smart install client active"
50.238.151.242  4786            Cisco Smart Install Client active
```

```
192.100.201.249 4786            Cisco Smart Install Client active
38.143.52.1     4786            Cisco Smart Install Client active
205.209.61.4    4786            Cisco Smart Install Client active
46.231.67.225   4786            Cisco Smart Install Client active
154.127.78.234  4786            Cisco Smart Install Client active
84.236.185.157  4786            Cisco Smart Install Client active
213.253.116.41  4786            Cisco Smart Install Client active
192.64.11.3     4786            Cisco Smart Install Client active
162.219.203.17  4786            Cisco Smart Install Client active
193.234.88.20   4786            Cisco Smart Install Client active
140.112.231.254 4786    router1.g4.ntu.edu.tw   Cisco Smart Install Client active
200.0.180.3     4786            Cisco Smart Install Client active
5.145.149.91    4786    ip-5-145-149-91.hosts.businesscomnetworks.com   Cisco Smart
Install Client active
152.32.162.254  4786            Cisco Smart Install Client active
130.127.172.129 4786            Cisco Smart Install Client active
<output omitted for brevity… there are hundreds of additional results>
```

Tip

Keep in mind that even though this is public information, you should not interact with any systems shown in Shodan results without permission from the owner. If the owner has a bug bounty program, you may get recognition and a reward for finding the affected system.

Amass, Maltego, and Other OSINT Tools

There are dozens of additional OSINT tools such as Amass and Maltego. Amass is an advanced open-source tool that is used to help information security researchers in the process of external asset discovery and OSINT. It is mainly utilized to discover names and information associated with businesses and organizations on the web. Amass uses several techniques to find subdomains associated with the target domain. These techniques may include DNS brute-forcing, reverse DNS sweeping, and more. It maps out the network infrastructure by identifying IP addresses, autonomous system numbers (ASNs), and netblocks related to the target. It can help identify assets that are exposed to the Internet and that might be associated with the target organization.

DNS sweeping is a technique used in network reconnaissance that involves querying DNS records across a range of IP addresses or domain names to gather information about the associated hosts. There are two primary forms of DNS sweeping:

- **Forward DNS sweeping**: A list of domain names is queried to resolve their corresponding IP addresses. This technique helps map out the IP addresses associated with different domains, which can be useful for identifying the infrastructure of a target organization or finding related services.
- **Reverse DNS sweeping**: A range of IP addresses is queried to identify the domain names (or hostnames) associated with those IPs. This technique is particularly useful for discovering hidden or unlisted domains and services that may not be directly visible through traditional means. In this process, the PTR (Pointer) records of IP addresses are queried to retrieve the domain names linked to them.

Amass integrates with a number of different data sources, including certificate transparency logs, whois databases, social networks, and DNS databases, to gather a large amount of information. It can create network maps and graphs that can help in visualizing the network structure of the target. It supports scripting, which can be used to automate various tasks and extend its functionality.

Maltego is a data visualization and analysis tool used for gathering and analyzing information in the fields of cybersecurity and forensic investigations. It is often used for OSINT as well. Maltego has a graphical interface where data can be visualized in a graph format, making it easier to identify patterns and relationships between data points. In Maltego, information is represented as entities. These entities can be anything like a person's name, a phone number, or an IP address.

Note

You can find additional information about Maltego and a few examples in my GitHub repository at **https://github.com/The-Art-of-Hacking/h4cker/blob/master/recon/maltego.md**.

The core functionality of Maltego revolves around transforms, which are scripts that can manipulate and analyze data in various ways. Transforms can be used to gather more information about entities, find relationships between them, and so on. Maltego can integrate with a wide range of data sources to gather information, including social networks, public records, websites, and more. Users can create their own entities and transforms, allowing for a high degree of customization to suit specific needs. It has a

wide range of applications including cyber threat intelligence, incident response, criminal investigations, and more.

Using Generative AI and the Gorilla Large Language Model to Interact with Tools Like Amass

The University of California, Berkeley, in collaboration with Microsoft, unveiled *Gorilla*, an advanced model based on the Llama framework, reputed to surpass GPT-4 in generating API calls. A notable attribute of Gorilla is its merger with a document retriever, enhancing its capability to effortlessly adapt to changes in documents during the testing phase. This flexibility is pivotal, especially when you're navigating through continuously updating API documentation and versions. Moreover, Gorilla significantly mitigates the issue of hallucination, a common obstacle faced when utilizing language models directly for prompts.

The team created *APIBench*, a comprehensive dataset that includes APIs from leading platforms such as HuggingFace, TorchHub, and TensorHub. Gorilla's exemplary performance highlights the substantial promise held by this kind of large language model and its applications. This fusion not only guarantees more accurate tool usage but also ensures staying abreast with the ever-changing documentation. For individuals keen on delving deeper into Gorilla, the models and relevant code are available at **https://github.com/ShishirPatil/gorilla**. Further details and the research paper can be found at **https://gorilla.cs.berkeley.edu**.

You can install the Gorilla CLI easily by using the **pip3 install gorilla-cli** command. Once it is installed, you can use prompts like the one shown in **Example 9-27**.

Example 9-27 Using Generative AI and the Gorilla LLM to Perform Passive Recon

Click here to view code image

```
┌──(omar㉿websploit)-[~]
└─$ gorilla "use Amass to perform recon of the secretcorp.org domain"
🦍  Welcome to Gorilla. Use arrows to select
   amass enum -d secretcorp.org | while read domain; do dig axfr +short "$domain" |
awk
   kubectl recon secretcorp.org
 » amass enum -d secretcorp.org
   : #Do nothing
```

The Gorilla CLI provides several options. Some options may be incorrect. However, in many cases the output is impressively good. The highlighted line in **Example 9-27** shows the correct option. It uses Amass to correctly enumerate the **secretcorp.org** domain.

Note

Chapter 11, "**Automating a Bug Hunt and Leveraging the Power of AI**," covers many other examples on how to use generative AI to learn and automate different tasks in a bug hunt during a bug bounty engagement.

Using Open Interpreter to Interact with Recon Tools

Open Interpreter is an open-source tool designed to enable developers to work with large language models (LLMs) on their local machines. It provides a natural language interface for interacting with a computer's general-purpose capabilities, such as running code, analyzing data, controlling web browsers, and creating or editing media files like photos and videos. Unlike hosted solutions like OpenAI's Code Interpreter, Open Interpreter runs locally, offering greater flexibility and control over runtime, file size, and Internet access.

Open Interpreter operates on the user's local machine, providing full access to the Internet and enabling the use of any required package or library without restrictions on runtime or file size. It supports tasks such as data analysis, GUI control, browser automation, media editing, and dataset management. It also includes vision capabilities for image analysis. Open Interpreter works with different LLMs, allowing users to select models like OpenAI models, Anthropic Claude models, as well as local open weight models. It also supports local model providers such as Ollama and LM Studio.

The installation process is extremely easy. You can install Open Interpreter using Python's pip package manager with the **pip install open-interpreter** command.

Note

A virtual environment is recommended for managing dependencies.

To start a session through the terminal, use the **interpreter** command and then start typing your prompts. You can execute code directly within the tool or integrate it into Python applications for custom functionality.

Example 9-28 includes an example of how to use Open Interpreter in Python to perform passive reconnaissance of the domain **secretcorp.org**.

Example 9-28 Using Open Interpreter in Python to Perform Passive Recon

Click here to view code image

```
from interpreter import interpreter

# Set the LLM model to use
interpreter.llm.model = "gpt-4o-mini"

# Perform passive reconnaissance on the target domain
interpreter.chat("Use Amass to perform passive reconnaissance on secretcorp.org.
Analyze the output. Save the output and analysis to a file called secretcorp.md.")

# Start an interactive chat
interpreter.chat()
```

This script is also available at my GitHub repository at **https://github.com/The-Art-of-Hacking/h4cker/tree/master/ai_research/open-interpreter-examples**.

Open Interpreter is a simple agent that creates its own prompts and plans of action to perform the tasks you requested. **Example 9-29** shows the "plan of action" created by Open Interpreter.

Example 9-29 The "Plan of Action" created by Open Interpreter

Click here to view code image

```
To perform passive reconnaissance using Amass on the domain secretcorp.org, I'll
follow these steps:
 1 Install Amass if it's not already installed.
 2 Run Amass to gather information about the domain.
 3 Analyze the output to gather insights.
 4 Save the output and the analysis to a file called secretcorp.md.
Let's start with the first step: checking if Amass is installed and installing it
if necessary.
  amass -version
Amass is already installed (version 4.2.0). Now, I'll proceed to run Amass to
gather passive reconnaissance data for the domain secretcorp.org. Let's execute
```

```
that command.
  amass enum -passive -d secretcorp.org -o secretcorp_output.txt
```

After Open Interpreter creates the plan of action and excutes the commands shown in **Example 9-29**, it performs the analysis of Amass's output and saves that analysis to a file. The contents of the file created by Open Interpreter is shown in **Example 9-30**. This file is also available in my GitHub repository at **https://github.com/The-Art-of-Hacking/h4cker/tree/master/ai_research/open-interpreter-examples**.

Example 9-30 The Contents of the File Created by Open Interpreter

Click here to view code image

```
### Passive Reconnaissance Analysis of secretcorp.org
The passive reconnaissance using Amass on secretcorp.org revealed the following:
- Unique Domains: 16
- Unique IP Addresses: 1
## Amass Output:
secretcorp.org (FQDN) --> ns_record --> ns-cloud-b3.googledomains.com (FQDN)
secretcorp.org (FQDN) --> ns_record --> ns-cloud-b1.googledomains.com (FQDN)
secretcorp.org (FQDN) --> ns_record --> ns-cloud-b2.googledomains.com (FQDN)
secretcorp.org (FQDN) --> ns_record --> ns-cloud-b4.googledomains.com (FQDN)
secretcorp.org (FQDN) --> mx_record --> mxb.mailgun.org (FQDN)
secretcorp.org (FQDN) --> mx_record --> mxa.mailgun.org (FQDN)
secretcorp.org (FQDN) --> node --> finance-app.secretcorp.org (FQDN)
secretcorp.org (FQDN) --> node --> backdoor.secretcorp.org (FQDN)
secretcorp.org (FQDN) --> node --> vpn.secretcorp.org (FQDN)
secretcorp.org (FQDN) --> node --> cloud.secretcorp.org (FQDN)
finance-app.secretcorp.org (FQDN) --> cname_record --> santosomar.github.io (FQDN)
backdoor.secretcorp.org (FQDN) --> cname_record --> santosomar.github.io (FQDN)
vpn.secretcorp.org (FQDN) --> cname_record --> santosomar.github.io (FQDN)
cloud.secretcorp.org (FQDN) --> cname_record --> santosomar.github.io (FQDN)
secretcorp.org (FQDN) --> node --> app1.secretcorp.org (FQDN)
secretcorp.org (FQDN) --> node --> internal.secretcorp.org (FQDN)
app1.secretcorp.org (FQDN) --> cname_record --> santosomar.github.io (FQDN)
ns-cloud-b2.googledomains.com (FQDN) --> a_record --> 216.239.34.107 (IPAddress)
ns-cloud-b2.googledomains.com (FQDN) --> aaaa_record --> 2001:4860:4802:34::6b
(IPAddress)
ns-cloud-b4.googledomains.com (FQDN) --> a_record --> 216.239.38.107 (IPAddress)
ns-cloud-b4.googledomains.com (FQDN) --> aaaa_record --> 2001:4860:4802:38::6b
(IPAddress)
internal.secretcorp.org (FQDN) --> cname_record --> santosomar.github.io (FQDN)
216.239.32.0/20 (Netblock) --> contains --> 216.239.34.107 (IPAddress)
216.239.32.0/20 (Netblock) --> contains --> 216.239.38.107 (IPAddress)
15169 (ASN) --> managed_by --> GOOGLE - Google LLC (RIROrganization)
15169 (ASN) --> announces --> 216.239.32.0/20 (Netblock)
```

```
ns-cloud-b3.googledomains.com (FQDN) --> a_record --> 216.239.36.107 (IPAddress)
ns-cloud-b3.googledomains.com (FQDN) --> aaaa_record --> 2001:4860:4802:36::6b
(IPAddress)
ns-cloud-b1.googledomains.com (FQDN) --> a_record --> 216.239.32.107 (IPAddress)
ns-cloud-b1.googledomains.com (FQDN) --> aaaa_record --> 2001:4860:4802:32::6b
(IPAddress)
216.239.32.0/20 (Netblock) --> contains --> 216.239.32.107 (IPAddress)
216.239.32.0/20 (Netblock) --> contains --> 216.239.36.107 (IPAddress)
secretcorp.org (FQDN) --> a_record --> 185.199.111.153 (IPAddress)
secretcorp.org (FQDN) --> a_record --> 185.199.110.153 (IPAddress)
secretcorp.org (FQDN) --> a_record --> 185.199.108.153 (IPAddress)
secretcorp.org (FQDN) --> a_record --> 185.199.109.153 (IPAddress)
secretcorp.org (FQDN) --> node --> mail.secretcorp.org (FQDN)
mail.secretcorp.org (FQDN) --> cname_record --> santosomar.github.io (FQDN)
2001:4860::/32 (Netblock) --> contains --> 2001:4860:4802:32::6b (IPAddress)
2001:4860::/32 (Netblock) --> contains --> 2001:4860:4802:34::6b (IPAddress)
2001:4860::/32 (Netblock) --> contains --> 2001:4860:4802:36::6b (IPAddress)
2001:4860::/32 (Netblock) --> contains --> 2001:4860:4802:38::6b (IPAddress)
185.199.108.0/22 (Netblock) --> contains --> 185.199.111.153 (IPAddress)
185.199.108.0/22 (Netblock) --> contains --> 185.199.110.153 (IPAddress)
185.199.108.0/22 (Netblock) --> contains --> 185.199.108.153 (IPAddress)
185.199.108.0/22 (Netblock) --> contains --> 185.199.109.153 (IPAddress)
15169 (ASN) --> announces --> 2001:4860::/32 (Netblock)
54113 (ASN) --> managed_by --> FASTLY - Fastly (RIROrganization)
54113 (ASN) --> announces --> 185.199.108.0/22 (Netblock)
secretcorp.org (FQDN) --> node --> sslvpn.secretcorp.org (FQDN)
sslvpn.secretcorp.org (FQDN) --> cname_record --> ext-sq.squarespace.com (FQDN)
mxa.mailgun.org (FQDN) --> a_record --> 34.149.236.64 (IPAddress)
34.144.0.0/13 (Netblock) --> contains --> 34.149.236.64 (IPAddress)
15169 (ASN) --> announces --> 34.144.0.0/13 (Netblock)
ext-sq.squarespace.com (FQDN) --> a_record --> 198.49.23.144 (IPAddress)
ext-sq.squarespace.com (FQDN) --> a_record --> 198.185.159.144 (IPAddress)
ext-sq.squarespace.com (FQDN) --> a_record --> 198.185.159.145 (IPAddress)
ext-sq.squarespace.com (FQDN) --> a_record --> 198.49.23.145 (IPAddress)
198.49.23.0/24 (Netblock) --> contains --> 198.49.23.144 (IPAddress)
198.49.23.0/24 (Netblock) --> contains --> 198.49.23.145 (IPAddress)
198.185.159.0/24 (Netblock) --> contains --> 198.185.159.144 (IPAddress)
198.185.159.0/24 (Netblock) --> contains --> 198.185.159.145 (IPAddress)
53831 (ASN) --> managed_by --> SQUARESPACE - Squarespace, Inc. (RIROrganization)
53831 (ASN) --> announces --> 198.49.23.0/24 (Netblock)
53831 (ASN) --> announces --> 198.185.159.0/24 (Netblock)
mxb.mailgun.org (FQDN) --> a_record --> 34.160.157.95 (IPAddress)
34.160.0.0/13 (Netblock) --> contains --> 34.160.157.95 (IPAddress)
15169 (ASN) --> announces --> 34.160.0.0/13 (Netblock)
```

The scenario demonstrated in **Examples 9-28**, 9-29, and 9-30 are straightforward, but they effectively demonstrate the immense power and versatility of tools like Open Interpreter for accomplishing these tasks.

Performing Active Reconnaissance

As mentioned earlier, with each step of the information-gathering phase, the goal is to gather additional information about the target. The act of gathering this information is called *enumeration*. So, let's talk about what kind of enumeration you would typically be doing in a penetration test. In an earlier example, we looked at the enumeration of hosts exposed to the Internet by **h4cker.org**. External enumeration of hosts is usually one of the first things you would do in a penetration test. Determining the Internet-facing hosts of a target network can help you identify the systems that are most exposed. Obviously, having a device that is publicly accessible over the Internet opens it up to attack from malicious actors all over the world. After you identify those systems, you then need to identify which services are accessible. The device should be behind a firewall, allowing minimal exposure to the services it is running. Sometimes, however, services that are not expected are exposed. To determine this, you can run a port scan to enumerate the services that are running on the exposed hosts.

A *port scan* is an active scan in which the scanning tool sends different types of probes to the target IP address and then examines the responses to determine whether the service is actually listening. For instance, with an Nmap SYN scan, the tool sends a TCP SYN packet to the TCP port it is probing. This process is also referred to as *half-open scanning* because it does not open a full TCP connection. If the response is a SYN/ACK, this would indicate that the port is actually in a listening state. If the response to the SYN packet is an RST (reset), this would indicate that the port is closed or not in a listening state. If the SYN probe does not receive any response, Nmap marks it as filtered because it cannot determine if the port is open or closed. **Example 9-31** shows the output of a SYN scan.

Example 9-31 Nmap SYN Scan Sample Output

Click here to view code image

```
┌──(root㉿websploit)-[~]
└─# nmap -sS 10.6.6.23
Starting Nmap 7.94 ( https://nmap.org )
Nmap scan report for 10.6.6.23
Host is up (0.000010s latency).
Not shown: 994 closed tcp ports (reset)
PORT    STATE SERVICE
21/tcp  open  ftp
22/tcp  open  ssh
```

```
53/tcp  open  domain
80/tcp  open  http
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds
MAC Address: 02:42:0A:06:06:17 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.23 seconds
```

Example 9-31 shows how to run a TCP SYN scan using Nmap by specifying the **-sS** option against a host with the IP address 192.168.88.251. As you can see, this system has several ports open. In some situations, you will want to use the many different Nmap options in your scans to get the results you're looking for. The sections that follow take a look at some of the most common options and types of scans available in Nmap.

Nmap allows you to adjust the timing of your scans to trade off between speed and being detected by a security operations center (SOC) tool. For example, you can use

- **-T0** (Paranoid): This option makes the scan very slow to evade detection by an intrusion detection systems (IDS).
- **-T1** (Sneaky): This option is a bit faster than T0 but still quite slow, aiming to avoid detection.
- **-T2** (Polite): This option slows down the scan to use less bandwidth and target machine resources.
- **-T3** (Normal): This option uses the default timing template that does not include any particular delays.
- **-T4** (Aggressive): This option increases the speed of the scan, potentially missing some targets or details.
- **-T5** (Insane): This option is extremely fast and likely to be detected by an IDS.

The Nmap Scripting Engine (NSE) is one of Nmap's most powerful and flexible features. It allows users to write (and share) simple scripts to automate a wide variety of networking tasks. Those scripts are executed in parallel, and the results are integrated into Nmap's regular output. These scripts can be used for more advanced network discovery, vulnerability detection, and exploitation. The following is a detailed explanation of different aspects of the NSE:

- Scripts are written in Lua, a powerful, efficient, lightweight, embeddable scripting language. These scripts define what actions will be performed against the target.

- The NSE includes a large number of libraries that provide additional functionality and allow script writers to easily create complex scripts without starting from scratch.
- Scripts are generally executed after the scanning phases (like port scanning) are completed, and they use the results of those phases to perform more advanced analysis.
- Nmap scripts are categorized into various groups, which help in identifying and using scripts for specific purposes. Some of the main categories are
  - **default**: Scripts that are safe to run and are executed when you use the **-sC** option or **--script=default**
  - **safe**: Scripts that do not affect the target network or application in any way
  - **intrusive**: Scripts that might have potential side effects on the target
  - **vuln**: Scripts that check for specific vulnerabilities
  - **exploit**: Scripts that attempt to exploit vulnerabilities
  - **auth**: Scripts that bypass authentication mechanisms
  - **discovery**: Scripts focused on network discovery
  - **version**: Scripts that determine versions of services
  - **brute**: Scripts that perform brute-force attacks

You can use NSE scripts by specifying the **--script** option followed by the script name or a category of scripts. For example:

Click here to view code image

```
nmap --script=http-title --script-args="http.useragent=Mozilla" 10.6.6.23
```

Users can write their custom scripts using the Lua programming language. A custom script generally contains the following sections:

- **Script Header**: Contains metadata like script name, description, categories, and so on
- **Rule Function**: Defines when the script should be executed
- **Action Function**: Contains the main logic of the script

For debugging and development of scripts, NSE provides various verbose and debug levels that can be adjusted using **-d** and **-v** options, respectively.

Tip

This book is not about Nmap. To learn more, you can get numerous examples of how to use Nmap, along with different exercises, at my GitHub repository at **https://hackerrepo.org**.

Creating Your Own Scanner Using Python

Creating your own scanner using Python involves utilizing libraries such as **socket** and **scapy** to craft and send packets to network hosts and then analyze the responses. Initially, you'd define the target IP address and the range of ports to scan. Using the **socket** library, you can create a connection to each port and check whether it is open or closed based on the response received. For more advanced scanning features, the **scapy** library can be employed to craft custom packets, enabling you to perform tasks such as ARP scanning, SYN scanning, and more. Integrating multithreading can optimize your scanner by allowing simultaneous scanning of multiple ports, thus speeding up the process significantly. Once your script is ready, you can further enhance it with functionalities such as logging results to a file or adding a user interface for easier use.

Tip

Scapy is a powerful Python library and interactive tool that allows you to send, sniff, dissect, and forge network packets. Its versatility enables users to create various types of packets from scratch, manipulate packet fields, and conduct detailed analyses of network traffic. **Scapy** is often used for network discovery, security testing (including penetration testing), and prototyping network protocols. It supports a wide range of protocols and can be extended to support additional protocols as required. It's a valuable tool in the field of network security and is used by security professionals, network administrators, and researchers to analyze and test network behaviors.

Example 9-32 shows a Python script that uses **Scapy** to perform a basic TCP port scan.

Example 9-32 Using *Scapy* to Perform a TCP Port Scan

Click here to view code image

```
from scapy.all import *
import sys

def tcp_port_scan(target, ports):
    for port in ports:
        tcp_packet = IP(dst=target) / TCP(dport=port, flags="S")
        response = sr1(tcp_packet, timeout=2, verbose=0)

        if response is not None and response[TCP].flags == 18:
            print(f"Port {port} is open on {target}")
        else:
            print(f"Port {port} is closed on {target}")

if __name__ == "__main__":
    target = sys.argv[1]
    ports = range(1, 1024)

    tcp_port_scan(target, ports)
```

Here's how to use the script:

1. Save the script in a file named port_scan.py.
2. Run the script by using the following command in the terminal:
   Click here to view code image

   ```
   python port_scan.py <target_ip>
   ```

3. The script begins by importing necessary modules:
   1. **from scapy.all import *** imports all necessary components from the **Scapy** library, a powerful interactive packet manipulation tool.
   2. **import sys** imports the system-specific parameters and functions module.
   3. The **tcp_port_scan(target, ports)** function is defined to perform the TCP port scan.
   4. For each port in the provided ports, it creates a TCP packet with the **S** (SYN) flag set using **IP(dst=target) / TCP(dport=port, flags="S")**.
   5. The script then sends the packet to the target machine using the **sr1()** function, which sends the packet and returns the first response received.
   6. If a response is received (the response is not **None**) and the TCP flags of the response are equal to **18** (**response[TCP].flags** == **18**), the script prints that the port is open. TCP flag **18** represents the SYN/ACK packet, which is usually the response to the SYN packet

when a port is open. If there is no response or the response is not SYN/ACK, the script prints that the port is closed.

7. In the **__main__** part of the script, it does the following:
8. **target** is set to the first argument given in the command line (**sys.argv[1]**), which is the IP address of the target machine.
9. **ports** is set to the range of 1–1023, which are the well-known port numbers.
10. The **tcp_port_scan()** function is then called with the target and ports as parameters.

This simple script does not handle many edge cases. In a real-world situation, additional code would be required to handle potential exceptions, timeouts, and other situations. This script and many other examples are available in my GitHub repository at **https://github.com/The-Art-of-Hacking/h4cker/tree/master/programming_and_scripting_for_cybersecurity**.

Exploring the Different Types of Enumeration

Next, we'll cover enumeration techniques that should be performed in the information-gathering phase of a penetration test. You will learn how and when these enumeration techniques should be used. The following sections also include examples of performing these types of enumeration by using Nmap.

Host Enumeration

The enumeration of hosts is one of the first tasks you need to perform in the information-gathering phase of a penetration test. Host enumeration is performed internally and externally. When it is performed externally, you typically want to limit the IP addresses you are scanning to just the ones that are part of the scope of the test. Doing so reduces the chance of inadvertently scanning an IP address that you are not authorized to test. When performing an internal host enumeration, you typically scan the full subnet or subnets of IP addresses being used by the target. Host enumeration is usually performed using a tool such as Nmap or Masscan; however, vulnerability scanners also perform this task as part of their automated testing. In earlier versions of Nmap, the **-sn** option was **-sP**. For additional information about the Nmap host discovery capabilities, visit **https://nmap.org/book/man-host-discovery.html**.

User Enumeration

Gathering a valid list of users is the first step in cracking a set of credentials. When you have the username, you can then begin brute-force attempts to get the account password. You perform user enumeration when you have gained access to the internal network. On a Windows network, you can do this by manipulating the Server Message Block (SMB) protocol, which uses TCP port 445. **Figure 9-5** shows how a typical SMB implementation works.

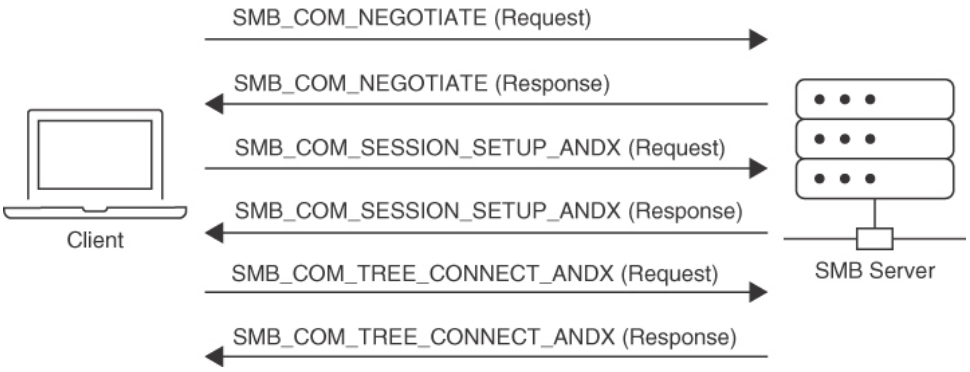


Figure 9-5

SMB Message Implementation

The information contained in the responses to these messages enables you to reveal information about the server:

- **SMB_COM_NEGOTIATE:** This message allows the client to tell the server what protocols, flags, and options it would like to use. The response from the server is also an SMB_COM_NEGOTIATE message. This response relays to the client which protocols, flags, and options it prefers. This information can be configured on the server itself. A misconfiguration sometimes reveals information that you can use in penetration testing. For instance, the server might be configured to allow messages without signatures. You can determine if the server is using share- or user-level authentication mechanisms and whether the server allows plaintext passwords. The response from the server also provides you with additional information, such as the time and time zone the server is using. This is necessary information for many penetration testing tasks.
- **SMB_COM_SESSION_SETUP_ANDX:** After the client and server have negotiated the protocols, flags, and options they will use for communication, the authentication process begins. Authentication is the primary function of the SMB_COM_SESSION_SETUP_ANDX message. The information sent in this message includes the client username, pass-

word, and domain. If this information is not encrypted, it is easy to sniff it right off the network. Even if it is encrypted, if the mechanism being used is not sufficient, the information can be revealed using simple tools (such as Lanman and NTLM in the case of Microsoft). An example of this being utilized is the **smb-enum-users.nse** script as shown here:

Click here to view code image

```
nmap --script smb-enum-users.nse <host>
```

Example 9-33 shows the results of the Nmap **smb-enum-users** script run against the target 192.168.88.251. As you can see, the results indicate that the script was able to enumerate the users that are configured on this Windows target.

Example 9-33 Enumerating SMB Users

Click here to view code image

```
┌──[root@websploit]─[~]
└── #nmap  --script smb-enum-users.nse 192.168.88.251
Starting Nmap 7.91 ( https://nmap.org ) at 2024-06-22 11:14 EDT
Nmap scan report for 192.168.88.251
Host is up (0.012s latency).
Not shown: 992 closed ports
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
3306/tcp open  mysql
8888/tcp open  sun-answerbook
9000/tcp open  cslistener
9090/tcp open  zeus-admin
Host script results:
| smb-enum-users:
|   VULNHOST-1\derek (RID: 1000)
|     Full name:
|     Description:
|_    Flags:       Normal user account
Nmap done: 1 IP address (1 host up) scanned in 0.81 seconds
```

The highlighted line in Example 9-33 reveals the user that was enumerated by Nmap (in this case, derek).

Group Enumeration

For a penetration tester, enumerating groups is helpful in determining the authorization roles that are being used in the target environment. The Nmap NSE script for enumerating SMB groups is **smb-enum-groups**. This script attempts to pull a list of groups from a remote Windows machine. You can also reveal the list of users that are members of those groups. The syntax of the command is as follows:

Click here to view code image

```
nmap --script smb-enum-groups.nse -p445 <host>
```

Example 9-34 displays the sample output of this command run against another Windows server (192.168.56.3). This example uses known credentials to gather information.

Example 9-34 Enumerating SMB Groups

Click here to view code image

```
┌─[root@websploit]─[~]
└── #nmap --script smb-enum-groups.nse --script-args
smbusername=vagrant,smbpass=vagrant 192.168.56.3
Starting Nmap 7.91 ( https://nmap.org )
Nmap scan report for 192.168.56.3
Host is up (0.0062s latency).
Not shown: 979 closed ports
PORT      STATE SERVICE
22/tcp    open  ssh
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
3306/tcp  open  mysql
3389/tcp  open  ms-wbt-server
MAC Address: 08:00:27:1B:A4:60 (Oracle VirtualBox virtual NIC)
Host script results:
| smb-enum-groups:
|   Builtin\Administrators (RID: 544): Administrator, vagrant, sshd_server
|   Builtin\Users (RID: 545): vagrant, sshd, sshd_server, leia_organa,
luke_skywalker, han_solo, artoo_detoo, c_three_pio, ben_kenobi, darth_vader,
anakin_skywalker, jarjar_binks, lando_calrissian, boba_fett, jabba_hutt, greedo,
chewbacca, kylo_ren
|   Builtin\Guests (RID: 546): Guest, ben_kenobi
|   Builtin\Power Users (RID: 547): boba_fett
|   Builtin\Print Operators (RID: 550): jabba_hutt
```

```
|   Builtin\Backup Operators (RID: 551): leia_organa
|   Builtin\Replicator (RID: 552): chewbacca
|   Builtin\Remote Desktop Users (RID: 555): greedo
|   Builtin\Network Configuration Operators (RID: 556): anakin_skywalker
|   Builtin\Performance Monitor Users (RID: 558): lando_calrissian
|   Builtin\Performance Log Users (RID: 559): jarjar_binks
|   Builtin\Distributed COM Users (RID: 562): artoo_detoo
|   Builtin\IIS_IUSRS (RID: 568): darth_vader
|   Builtin\Cryptographic Operators (RID: 569): han_solo
|   Builtin\Event Log Readers (RID: 573): c_three_pio
|   Builtin\Certificate Service DCOM Access (RID: 574): luke_skywalker
|_  VAGRANT-2008R2\WinRMRemoteWMIUsers__ (RID: 1003): <empty>
Nmap done: 1 IP address (1 host up) scanned in 0.81 seconds
┌─[root@websploit]─[~]
└── #
```

The highlighted output in **Example 9-34** shows the enumerated groups and users in the target host.

Network Share Enumeration

Identifying systems on a network that are sharing files, folders, and printers is helpful in building out an attack surface of an internal network. The Nmap **smb-enum-shares** NSE script uses Microsoft Remote Procedure Call (MSRPC) to retrieve information about remote shares. The syntax of the Nmap **smb-enum-shares.nse** script is as follows:

Click here to view code image

```
nmap --script smb-enum-shares.nse -p 445 <host>
```

Example 9-35 demonstrates the enumeration of SMB shares.

Example 9-35 Enumerating SMB Shares

Click here to view code image

```
┌─[root@websploit]─[~]
└── #nmap --script smb-enum-shares.nse -p 445 192.168.88.251
Starting Nmap 7.91 ( https://nmap.org ) at 2024-06-22 11:27 EDT
Nmap scan report for 192.168.88.251
Host is up (0.0011s latency).

PORT    STATE SERVICE
445/tcp open  microsoft-ds
```

```
Host script results:
| smb-enum-shares:
|   account_used: guest
<output omitted for brevity>
|     Users: 0
|     Max Users: <unlimited>
|     Path: C:\var\lib\samba\printers
|     Anonymous access: <none>
|     Current user access: <none>
|   \\192.168.88.251\secret_folder:
|     Type: STYPE_DISKTREE
|     Comment: Extremely sensitive information
|     Users: 0
|     Max Users: <unlimited>
|     Path: C:\secret_folder
|     Anonymous access: <none>
|_    Current user access: <none>

Nmap done: 1 IP address (1 host up) scanned in 0.39 seconds
┌──[root@websploit]─[~]
└── #
```

Additional SMB Enumeration Examples

The system used in earlier examples (with the IP address 192.168.88.251) is running Linux and Samba. However, it is not easy to determine that it was a Linux system from the results of previous scans. An easy way to perform additional enumeration and fingerprinting of the applications and operating system running on a host is by using the **nmap -sC** command. The **-sC** option runs the most common Nmap Scripting Engine scripts based on the ports that were found to be open on the target system.

Tip

You can locate the installed NSE scripts in Kali Linux and Parrot Security OS by simply using the **locate *.nse** command. For a detailed explanation of the NSE and how to create new scripts using the Lua programming language, see **https://nmap.org/book/man-nse.html**.

Example 9-36 shows the output of the **nmap -sC** command launched against the Linux system running Samba (192.168.88.251).

Example 9-36 Running the Nmap NSE Default Scripts

Click here to view code image

```
┌─[root@websploit]─[~]
└── #nmap -sC 192.168.88.251
Starting Nmap 7.80 ( https://nmap.org ) at 2024-06-21 17:38 EDT
Nmap scan report for 192.168.88.251
Host is up (0.00011s latency).
Not shown: 992 closed ports
PORT     STATE SERVICE
22/tcp   open  ssh
| ssh-hostkey:
|   2048 d0:0c:83:4d:7f:84:2c:60:96:9f:df:26:da:d2:11:9a (RSA)
|   256 e2:aa:69:ab:a3:e6:0f:13:c5:5a:65:f2:d5:16:8c:3e (ECDSA)
|_  256 21:4b:27:7b:6e:a6:d4:33:86:60:cb:39:3b:48:9c:0b (ED25519)
80/tcp   open  http
|_http-title: WebSploit Mayhem
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
3306/tcp open  mysql
| mysql-info:
|   Protocol: 10
|   Version: 5.5.47-0ubuntu0.14.04.1
|   Thread ID: 3
|   Capabilities flags: 63487
|   Some Capabilities: InteractiveClient, DontAllowDatabaseTableColumn,
FoundRows, IgnoreSigpipes, Support41Auth, ODBCClient, ConnectWithDatabase, Long-
Password, SupportsTransactions, IgnoreSpaceBeforeParenthesis, Speaks41ProtocolOld,
Speaks41ProtocolNew, SupportsCompression, SupportsLoadDataLocal, LongColumnFlag,
SupportsMultipleResults, SupportsMultipleStatments, SupportsAuthPlugins
|   Status: Autocommit
|   Salt: b_60.4ZH=52:l5ajmhBP
|_  Auth Plugin Name: mysql_native_password
8888/tcp open  sun-answerbook
9000/tcp open  cslistener
9090/tcp open  zeus-admin
MAC Address: 1E:BD:4F:AA:C6:BA (Unknown)
Host script results:
|_clock-skew: mean: 17s, deviation: 0s, median: 17s
|_nbstat: NetBIOS name: VULNHOST-1, NetBIOS user: <unknown>, NetBIOS MAC: <unknown>
(unknown)
| smb-os-discovery:
|   OS: Windows 6.1 (Samba 4.9.5-Debian)
|   Computer name: vulnhost-1
|   NetBIOS computer name: VULNHOST-1\x00
|   Domain name: ohmr.org
|   FQDN: vulnhost-1.ohmr.org
```

```
|_  System time: 2022-06-21T21:38:40+00:00
| smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|_  message_signing: disabled (dangerous, but default)
<output omitted for brevity>
```

The highlighted lines in **Example 9-33** show the details about the Samba version that is running on the system (Samba Version 4.9.5). You can also see that even though the OS is marked as Windows 6.1, the correct operating system is Debian.

Example 9-37 shows the output of the **samba -V** command at the target system (**vulnhost-1**), which confirms that the scanner was able to determine the correct Samba version.

Example 9-37 Confirming the Scan Results in the Target System

Click here to view code image

```
omar@vulnhost-1:~$ sudo samba -V
Version 4.9.5-Debian
omar@vulnhost-1:~$
```

You can also use tools such as **enum4linux** to enumerate Samba shares including user accounts, shares, and other configurations. **Example 9-38** shows the output of the **enum4linux** tool after it was launched against the target system (192.168.88.251).

Example 9-38 Enumerating Additional Information Using *enum4linux*

Click here to view code image

```
┌─ [root@websploit]─[~]
└── #enum4linux 192.168.88.251
Starting enum4linux v0.8.9 ( http://labs.portcullis.co.uk/application/enum4linux/ )
 ==========================
|    Target Information    |
 ==========================
Target ........... 192.168.88.251
RID Range ........ 500-550,1000-1050
Username ......... ''
Password ......... ''
Known Usernames .. administrator, guest, krbtgt, domain admins, root, bin, none
```

```
 ==========================================================
|    Enumerating Workgroup/Domain on 192.168.88.251    |
 ==========================================================
[+] Got domain/workgroup name: WORKGROUP
 ============================================
|    Nbtstat Information for 192.168.88.251    |
 ============================================
Looking up status of 192.168.88.251
        VULNHOST-1      <00> -         B <ACTIVE>  Workstation Service
        VULNHOST-1      <03> -         B <ACTIVE>  Messenger Service
        VULNHOST-1      <20> -         B <ACTIVE>  File Server Service
        ..__MSBROWSE__. <01> - <GROUP> B <ACTIVE>  Master Browser
        WORKGROUP       <00> - <GROUP> B <ACTIVE>  Domain/Workgroup Name
        WORKGROUP       <1d> -         B <ACTIVE>  Master Browser
        WORKGROUP       <1e> - <GROUP> B <ACTIVE>  Browser Service Elections
        MAC Address = 00-00-00-00-00-00
 =====================================
|    Session Check on 192.168.88.251    |
 =====================================
[+] Server 192.168.88.251 allows sessions using username '', password ''
 ============================================
|    Getting domain SID for 192.168.88.251    |
 ============================================
Domain Name: WORKGROUP
Domain Sid: (NULL SID)
[+] Can't determine if host is part of domain or part of a workgroup
 =====================================
|    OS information on 192.168.88.251    |
 =====================================
Use of uninitialized value $os_info in concatenation (.) or string at ./enum4linux.
pl line 464.
[+] Got OS info for 192.168.88.251 from smbclient:
[+] Got OS info for 192.168.88.251 from srvinfo:
          VULNHOST-1     Wk Sv PrQ Unx NT SNT Samba 4.9.5-Debian
          platform_id     : 500
          os version      : 6.1
          server type     : 0x809a03
 ==============================
|    Users on 192.168.88.251    |
 ==============================
index: 0x1 RID: 0x3e8 acb: 0x00000010 Account: derek   Name:  Desc:
user:[derek] rid:[0x3e8]
 ========================================
|    Share Enumeration on 192.168.88.251    |
 ========================================
          Sharename       Type      Comment
          ---------       ----      -------
          print$          Disk      Printer Drivers
          secret_folder   Disk      Extremely sensitive information
```

```
          IPC$            IPC       IPC Service (Samba 4.9.5-Debian)
SMB1 disabled -- no workgroup available
[+] Attempting to map shares on 192.168.88.251
//192.168.88.251/print$     Mapping: DENIED, Listing: N/A
//192.168.88.251/secret_folder     Mapping: DENIED, Listing: N/A
=====================================================
|    Password Policy Information for 192.168.88.251    |
 =====================================================
[+] Attaching to 192.168.88.251 using a NULL share
[+] Trying protocol 139/SMB...
[+] Found domain(s):
          [+] VULNHOST-1
          [+] Builtin
[+] Password Info for Domain: VULNHOST-1
          [+] Minimum password length: 5
          [+] Password history length: None
<output omitted for brevity>
=====================================================
|    Users on 192.168.88.251 via RID cycling (RIDS: 500-550,1000-1050)    |
=====================================================
[I] Found new SID: S-1-22-1
[I] Found new SID: S-1-5-21-2226316658-154127331-1048156596
[I] Found new SID: S-1-5-32
[+] Enumerating users using SID S-1-5-21-2226316658-154127331-1048156596 and logon
username '', password ''
<output omitted for brevity>
S-1-5-21-2226316658-154127331-1048156596-501 VULNHOST-1\nobody (Local User)
S-1-5-21-2226316658-154127331-1048156596-513 VULNHOST-1\None (Domain Group)
S-1-5-21-2226316658-154127331-1048156596-1000 VULNHOST-1\derek (Local User)
<output omitted for brevity>
[+] Enumerating users using SID S-1-22-1 and logon username '', password ''
S-1-22-1-1000 Unix User\omar (Local User)
S-1-22-1-1001 Unix User\derek (Local User)
[+] Enumerating users using SID S-1-5-32 and logon username '', password ''
<output omitted for brevity>
```

There is a Python-based enum4linux implementation called **enum4linux-ng** that can be downloaded from **https://github.com/cddmp/enum4linux-ng**.

Example 9-39 includes an example of the SMB enumeration using **enum4linux-ng**.

Example 9-39 Enumeration Using *enum4linux-ng*

Click here to view code image

```
┌─[root@websploit]─[~/enum4linux-ng]
└── #./enum4linux-ng.py -As 192.168.88.251
ENUM4LINUX - next generation

 ==========================
|    Target Information    |
 ==========================
[*] Target ........... 192.168.88.251
[*] Username ......... ''
[*] Random Username .. 'opaftohf'
[*] Password ......... ''
[*] Timeout .......... 5 second(s)

 =====================================
|    Service Scan on 192.168.88.251    |
 =====================================
[*] Checking LDAP
<output omitted for brevity>
|    OS Information via RPC on 192.168.88.251    |
 ================================================
[+] The following OS information was found:
server_type_string = Wk Sv PrQ Unx NT SNT Samba 4.9.5-Debian
platform_id        = 500
os_version         = 6.1
server_type        = 0x809a03
os                 = Linux/Unix (Samba 4.9.5-Debian)
 =====================================
|    Users via RPC on 192.168.88.251    |
 =====================================
[*] Enumerating users via 'querydispinfo'
[+] Found 2 users via 'querydispinfo'
[*] Enumerating users via 'enumdomusers'
[+] Found 2 users via 'enumdomusers'
[+] After merging user results we have 2 users total:
'1000':
  username: derek
  name: ''
  acb: '0x00000010'
  description: ''
'1001':
  username: omar
  name: ''
  acb: '0x00000010'
  description: ''
 =======================================
|    Groups via RPC on 192.168.88.251    |
 =======================================
[*] Enumerating local groups
```

```
[+] Found 0 group(s) via 'enumalsgroups domain'
[*] Enumerating builtin groups
[+] Found 0 group(s) via 'enumalsgroups builtin'
[*] Enumerating domain groups
[+] Found 0 group(s) via 'enumdomgroups'
 ========================================
|    Shares via RPC on 192.168.88.251    |
 ========================================
[*] Enumerating shares
[+] Found 3 share(s):
IPC$:
  comment: IPC Service (Samba 4.9.5-Debian)
  type: IPC
print$:
  comment: Printer Drivers
  type: Disk
secret_folder:
  comment: Extremely sensitive information
  type: Disk
[*] Testing share IPC$
[-] Could not check share: STATUS_OBJECT_NAME_NOT_FOUND
[*] <output omitted for brevity>
```

The highlighted lines in **Example 9-39** show the enumerated users, Samba version, and shared folders. You can also use simple tools such as **smbclient** to enumerate shares and other information from a system running SMB, as demonstrated in **Example 9-40**.

Example 9-40 Enumeration Using *smbclient*

Click here to view code image

```
┌─[root@websploit]
└── #smbclient -L \\\\192.168.88.251
          Sharename       Type      Comment
               ---------       ----      -------
          print$          Disk      Printer Drivers
          secret_folder   Disk      Extremely sensitive information
          IPC$            IPC       IPC Service (Samba 4.9.5-Debian)
SMB1 disabled -- no workgroup available
┌─[root@websploit]─[~/enum4linux-ng]
└── #
```

Web Page Enumeration/Web Application Enumeration

Once you have identified that a web server is running on a target host, the next step is to take a look at the web application and begin to map out the attack surface. You can map out the attack surface of a web application in a few different ways. The handy Nmap tool actually has an NSE script available for brute-forcing the directory and file paths of web applications. Armed with a list of known files and directories used by common web applications, it probes the server for each of the items on the list. Based on the response from the server, it can determine whether those paths exist. This capability is handy for identifying things, like the Apache or Tomcat default manager page, that are commonly left on web servers and can be potential paths for exploitation.

The syntax of the **http-enum** NSE script is as follows:

Click here to view code image

```
nmap -sV --script=http-enum <target>
```

Example 9-41 displays the results of running the script against the host with the IP address 192.168.88.251.

Example 9-41 Sample Nmap *http-enum* Script Output

Click here to view code image

```
┌─[root@websploit]─[~]
└── #nmap -sV --script=http-enum -p 80 192.168.88.251
Starting Nmap 7.91 ( https://nmap.org ) at 2024-06-22 11:53 EDT
Nmap scan report for 192.168.88.251
Host is up (0.0011s latency).

PORT   STATE SERVICE VERSION
80/tcp open  http    nginx 1.17.2
| http-enum:
|   /admin/: Possible admin folder
|   /admin/index.html: Possible admin folder
|_  /s/: Potentially interesting folder
|_http-server-header: nginx/1.17.2
Service detection performed. Please report any incorrect results at https://
nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.54 seconds
┌─[root@websploit]─[~]
└── #
```

The highlighted output in **Example 9-38** shows several enumerated directories/folders and the version of the web server being used (nginix 1.17.2). This is a good place to start in attacking a web application.

Another web server enumeration tool we should talk about is Nikto. Nikto is an open-source web vulnerability scanner that has been around for many years. It's not as robust as the commercial web vulnerability scanners; however, it is very handy for running a quick script to enumerate information about a web server and the applications it is hosting. Because of the speed at which Nikto works to scan a web server, it is very noisy. It provides a number of options for scanning, including the capability to authenticate to a web application that requires a username and password. **Example 9-42** shows an example of the output of a Nikto scan being run against the same host.

Example 9-42 Sample Nikto Scan

Click here to view code image

```
┌─[root@websploit]─[~]
└── #nikto -h 192.168.88.251
- Nikto v2.1.6
---------------------------------------------------------------------------
+ Target IP:          192.168.88.251
+ Target Hostname:    192.168.88.251
+ Target Port:        80
---------------------------------------------------------------------------
+ Server: nginx/1.17.2
+ The anti-clickjacking X-Frame-Options header is not present.
+ The X-XSS-Protection header is not defined. This header can hint to the user
agent to protect against some forms of XSS
+ The X-Content-Type-Options header is not set. This could allow the user agent to
render the content of the site in a different fashion to the MIME type
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ OSVDB-3092: /admin/: This might be interesting...
+ /admin/index.html: Admin login page/section found.
+ /wp-admin/: Admin login page/section found.
+ /wp-login/: Admin login page/section found.
+ 7916 requests: 0 error(s) and 7 item(s) reported on remote host
+ End Time:           2024-06-22 11:57:59 (GMT-4) (15 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
┌─[root@websploit]─[~]
└── #
```

No tool is perfect! It is recommended that you become familiar with the behavior and output of different tools.

Using BloodHound for a Bug Bounty

In **Chapter 7**, "**Active Directory and Linux Environments**," you learned different techniques to attack Active Directory (AD). You learned about powerful tools like BloodHound and how it is widely used to identify and analyze security risks and potential pathways of attack within AD domains.

Using BloodHound in bug bounty programs can be a strategic approach, especially when the scope of the program includes complex environments with AD. First and foremost, ensure that the use of BloodHound is within the permissible scope of the bug bounty program. You can use BloodHound during the initial reconnaissance phase to gather detailed information about the AD environment. This could include identifying trusts, group memberships, permission assignments, and session information.

You can see many different examples and "recipes" about how to use BloodHound at **https://www.thehacker.recipes/ad/recon/bloodhound**.

BloodHound excels in identifying potential attack paths, such as privilege escalation and lateral movement opportunities. Use it to find hidden vulnerabilities that could be exploited to compromise the environment.

You can also leverage the data analysis and visualization capabilities of BloodHound to understand the complex relationships in the AD environment. This can help you spot patterns and connections that might not be evident through conventional means. BloodHound can help identify misconfigurations in the AD setup, such as excessive permissions or insecure trusts, which could be reported as potential vulnerabilities in the bug bounty program.

Tip

BloodHound uses graph theory to reveal the hidden and often unintended relationships within AD environments. It collects data from the target AD environments using various methods and techniques, such as LDAP enumeration, SMB/NetSession enumeration, and more. BloodHound provides a graphical interface where it visually represents the

relationships and permissions within the AD environment, making it easier for analysts to identify potential attack paths.

One of the primary uses of BloodHound is to identify potential attack paths that an attacker could exploit to escalate privileges or move laterally within the network. By identifying potential vulnerabilities and attack paths, BloodHound assists security professionals in strengthening the security posture of the AD environment by remedying identified issues.

BloodHound is commonly used in red team operations to simulate sophisticated attacks on AD environments; however, it also can be used in some bug bounty engagements.

BloodHound has three main components:

- **SharpHound**: The data collector component, which gathers a multitude of data types from the target environments.
- **BloodHound Interface**: The frontend interface where the data is visualized and analyzed. It provides various analytics and querying capabilities to understand complex relationships and identify attack paths.
- **Neo4j Database**: The backend database where collected data is stored. BloodHound uses the Neo4j graph database to manage and query relationship data effectively.

Packet Inspection and Eavesdropping

As a bug bounty hunter, you can use tools like Wireshark, **tshark**, and **tcpdump** to collect packet captures for packet inspection and eavesdropping. Anyone who has been involved with networking or security has at some point used these tools to capture and analyze traffic on a network. For a penetration tester, such tools can be convenient for performing passive reconnaissance. Of course, this type of reconnaissance requires either a physical or wireless connection to the target. If your concern is being detected, you are probably safer to attempt a wireless connection because this would not require you to be inside the building. Many times, a company's wireless footprint bleeds outside its physical walls. As a penetration tester, you then have the opportunity to potentially collect information about the target and possibly gain access to the network to sniff traffic.

Note

The majority of bug bounty programs focus on external-facing assets, such as websites, APIs, mobile apps, and public cloud services. These systems are accessible over the Internet and can be tested by ethical hackers from anywhere in the world. This makes sense because external systems are the most exposed to potential attackers, and companies want to ensure these systems are as secure as possible. External eavesdropping (such as intercepting data in transit) is generally not viable in these scenarios because ethical hackers are crossing the public Internet to reach the target. The use of encryption protocols like HTTPS ensures that sensitive data is protected during transmission, making eavesdropping difficult without directly compromising the target system.

Understanding the Art of Performing Vulnerability Scans

Once you have identified the target hosts that are available and the services that are listening on those hosts, you can then begin to probe those services to determine if there are any weaknesses; this is what vulnerability scanners do. Vulnerability scanners use a number of different methods to determine whether a service is vulnerable. The primary method is to identify the version of the software that is running on the open service and try to match it with an already-known vulnerability. For instance, if a vulnerability scanner determines that a Linux server is running an outdated version of the Apache web server that is vulnerable to remote exploitation, it reports that vulnerability as a finding.

Of course, the main concern with automated vulnerability scanners is false positives; the output from a vulnerability scan can sometimes be useless if there is no validation done on the findings. Turning over a report full of false positives to a developer or an administrator who is then responsible for fixing the issues can really cause conflicts. You don't want someone chasing down findings in your report just to find out that they are false positives.

Understanding the Types of Vulnerability Scans

The type of vulnerability scan to use is usually driven by scan policy that is created in the automated vulnerability scanning tool. Each tool has many options available for scanning. You can often just choose to do a full

scan that will operate all scanning options, although you might not be able to use every option (for instance, if you are scanning a production environment or a device that is prone to crashing when scanning occurs). In such situations, you must be careful to select only the scan options that are less likely to cause issues. The sections that follow take a closer look at the typical scan types.

Unauthenticated Scans

By default, vulnerability scanners do not use credentials to scan a target. If you provide only the IP address of the target and click Scan, the tool will begin enumerating the host from the perspective of an unauthenticated remote attacker. An *unauthenticated* scan shows only the network services that are exposed to the network. The scanner attempts to enumerate the ports open on the target host. If the service is not listening on the network segment that the scanner is connected to, or if it is firewalled, the scanner will report the port as closed and move on. However, this does not mean that there is not a vulnerability. Sometimes it is possible to access ports that are not exposed to the network via SSH port forwarding and other tricks. It is still important to run a credentialed scan when possible.

Authenticated Scans

In some cases, it is best to run a credentialed (or *authenticated*) scan against a target to get a full picture of the attack surface. An authenticated scan requires you to provide the scanner with a set of credentials that have root-level access to the system. The reason for this is that the scanner actually logs in to the target via SSH or some other mechanism. It then runs commands like **netstat** to gather information from inside the host. Many of the commands that the scanner runs require root-level access to be able to gather the correct information from the system.

Example 9-43 shows the **netstat** command run by a nonprivileged user (omar) and then run again by a root user in **Example 9-44**. You can see that the output is different for the different user-level permissions. Specifically, notice that when running as the user **omar**, the PID/Program name is not available, and when running as the user **root**, that information is displayed.

Example 9-43 *Netstat* Example Without Root-Level Access

Click here to view code image

```
omar@vulnhost-1 ~ % netstat -tunap
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
PID/Program name
tcp        0      0 0.0.0.0:9000            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:9001            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:9002            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:3306            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:139             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:8881            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:8882            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:8883            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:8884            0.0.0.0:*               LISTEN
<output omitted for brevity>
```

Example 9-44 shows the details about the process and program name associated with the opened port.

Example 9-44 *Netstat* Example with Root-Level Access

Click here to view code image

```
omar@vulnhost-1 ~ % sudo netstat -tunap
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address     Foreign Address   State     PID/Program name
tcp        0      0 0.0.0.0:9000      0.0.0.0:*         LISTEN    1076/docker-proxy
tcp        0      0 0.0.0.0:9001      0.0.0.0:*         LISTEN    762/docker-proxy
tcp        0      0 0.0.0.0:9002      0.0.0.0:*         LISTEN    1287/docker-proxy
tcp        0      0 0.0.0.0:3306      0.0.0.0:*         LISTEN
<output omitted for brevity>
```

Challenges to Consider When Running a Vulnerability Scan

The previous sections touched on a number of different issues that should factor into how you perform your scanning. The sections that follow go into further detail about some of the specific aspects that should be considered when building a scanning policy and actually performing scans.

Considering the Best Time to Run a Scan

The timing of when to run a scan is typically of most concern when you are scanning a production network. If you're scanning a device in a lab environment, there is normally not much concern because a lab environment is not being used by critical applications. There are a few reasons running a scan on a production network should be done carefully. First, the network traffic that is being generated by a vulnerability scan can and will cause a lot of noise on the network. It can also cause significant congestion, especially when your scans are traversing multiple network hops. (We'll talk about this further shortly.)

Another consideration in choosing a time to run a scan is the fact that many of the options or plug-ins that are performed in a vulnerability scan can and will crash the target device as well as the network infrastructure. For this reason, you should be sure that when scanning on a production network, you are scanning at times that will have less impact on end users and servers. Most of the time scanning in the early hours of the day, when no one is using a network for critical purposes, is best.

Determining What Protocols Are in Use

One of the first things you need to know about a network or target device before you begin running vulnerability scans is what actual protocols are being used. If a target device is using both Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) for services that are running, and you only run a vulnerability scan against TCP ports, then you are going to miss any vulnerabilities that might be found on the UDP services.

UDP scanning can indeed take significantly longer than TCP scanning and is more prone to false positives due to the inherent characteristics of UDP. Unlike TCP, which is a connection-oriented protocol that relies on a three-way handshake to establish communication, UDP is connectionless. This means that when a UDP packet is sent, there is no built-in mechanism to confirm whether the packet was received. As a result, scanners like Nmap must wait for a response or lack thereof to determine the status of a port. If a port is open, it may not send any response at all. If a port is closed, the target might send back an ICMP "port unreachable" message. If no response is received, the port could be either open or filtered (blocked by a firewall), making it difficult to distinguish between the two states. This lack of feedback causes scanners to rely on timeouts and retransmissions, which significantly slows down the scanning process.

Many systems implement rate limiting on ICMP error messages (such as "port unreachable"). For example, Linux systems often limit these messages to one per second. If you're scanning thousands of ports, this rate limiting can cause the scan to take hours or even days. Additionally, since UDP scans often receive no response from open ports, scanners must wait for long timeouts before concluding that a port is either open or filtered. This further extends the duration of the scan compared to TCP, where closed ports quickly respond with a TCP RST (reset) packet.

Network Topology

As mentioned previously, the network topology should always be taken into consideration when it comes to vulnerability scanning. Of course, scanning across a WAN connection is never recommended because it would significantly impact any of the devices along the path. The rule of thumb when determining where in the network topology to run a vulnerability scan is that it should always be performed as close to the target as possible. For example, if you are scanning a Windows server that is sitting inside your DMZ, the best location for your vulnerability scanner is adjacent to the server on the DMZ. By placing it there, you can eliminate any concerns about impacting devices that your scanner traffic is traversing.

Aside from the impact on the network infrastructure, another concern is that any device that you traverse could also affect the results of your scanner. This is mostly a concern when traversing a firewall device; in addition, other network infrastructure devices could possibly impact the results as well.

Bandwidth Limitations

Let's take a moment to consider the effects of bandwidth limitations on vulnerability scanning. Obviously, any time you flood a network with a bunch of traffic, it is going to cause an issue with the amount of bandwidth that is available. As a penetration testing professional, you need to be cognizant of how you are affecting the bandwidth of the networks or systems you're scanning. Specifically, depending on the amount of bandwidth you have between the scanner and the target, you might need to adjust your scanner settings to accommodate lower-bandwidth situations. If you're scanning across a VPN or WAN link that most likely has limited bandwidth, you will want to adjust your scanning options so that you are not causing bandwidth consumption issues. The settings that need to be

adjusted are typically those related to flooding and denial-of-service type attacks.

Query Throttling

To work around the issue of bandwidth limitations and vulnerability scanning, slowing down the traffic created by your scanner can often help. You can typically achieve this slowdown by modifying the options of the scanning policy. One way to do this is to reduce the number of attack threads that are being sent to the target at the same time. There isn't a specific rule of thumb for the number of threads. It really depends on the robustness of the target. Some targets are more fragile than others. Another way to accomplish this is to reduce the scope of the plug-ins/attacks that the scanner is checking for. If you know that the target device is a Linux server, you can disable the attacks for other operating systems, such as Windows. Even though the attacks won't work against the Linux server, it still needs to receive and respond to the traffic. This additional traffic can cause a bottleneck in processing and network traffic consumption. Limiting the number of requests that the target would need to respond to would reduce the risk of causing issues on the target such as crashing, thus resulting in a more successful scan.

Fragile Systems/Nontraditional Assets

When using a vulnerability scanner against your internal network, you must take into consideration the devices on the network that might not be able to stand up to the traffic that is hurled at it by a vulnerability scanner. For these systems, you might need to either adjust the scanning options to reduce the risk of crashing the device or completely exempt the specific device from being scanned. Unfortunately, by exempting the specific device, you reduce the overall security of the environment.

IoT devices are often considered "fragile systems." Historically, these devices have not been able to withstand vulnerability scanning attempts. With the surge in IoT devices, today many more devices may be considered fragile, and you need to consider them when planning for vulnerability scanning. The typical way to address fragile devices is to exempt them from a scan; however, these devices can pose a risk to the environment and do need to be scanned. To address this issue, you can "throttle" the scan frequency as well as the options used in the scan policy to reduce the likelihood of crashing the device.

Performing Web Application and API Recon

There are several techniques and tools to perform reconnaissance of modern web applications and APIs. You already learned about some of these techniques, such as subdomain enumeration using tools like Sublist3r and Amass to identify subdomains of the main domain that might host different applications and potentially have vulnerabilities. The following sections describe several other techniques and methodologies that can be followed to perform reconnaissance of web applications and APIs.

Directory and File Brute-Forcing

A step to expand your insight into the website or web app potential vulnerabilities is to force-test the directories of the identified web servers. Uncovering directories in servers is a crucial endeavor because it may lead you to concealed admin interfaces, setup files, credential files, deprecated features, database backups, and repository files. The act of directory force-testing can, on occasion, grant you complete control over a server.

Although you might not pinpoint immediate exploitable weaknesses, the data derived from directories often sheds light on the application's architecture and underlying technology. For instance, spotting a pathway that signifies "phpmyadmin" generally indicates that the application is PHP-based.

For the purpose of directory force-testing, tools like **Dirsearch** or **gobuster** can be employed. These applications generate URLs using wordlists and subsequently send requests to a web server. When the server returns a response code within the 200 range, it confirms the existence of the directory or file. Consequently, you can navigate to the page to explore the contents hosted by the application. A 404 response code denotes nonexistence of the directory or file, while a 403 indicates its existence but with restricted access. It is advisable to scrutinize 403 pages meticulously to determine if there's a possibility to circumvent the security measures and access the information housed therein.

The HTTP 301 status code, also known as "Moved Permanently," indicates that the requested resource has been permanently moved to a new URL. This status code is part of the 3xx class of HTTP responses, which are used for redirection. When a server responds with a 301 status code, it includes a Location header that specifies the new URL where the resource

can be found. Browsers and search engines will automatically follow this redirection to the new URL.

Tools like **gobuster**, **ffuf**, and **feroxbuster** can be used to perform this type of reconnaissance. **Example 9-45** shows how to use **gobuster** to perform directory enumeration.

Example 9-45 Using *gobuster* to Perform Directory Enumeration

Click here to view code image

```
┌──(root㉿websploit)-[~]
└─# gobuster dir -w words.txt -u http://10.6.6.23
===============================================================
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://10.6.6.23
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /root/words.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.6
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
/s                    (Status: 301) [Size: 185] [--> http://10.6.6.23/s/]
/admin                (Status: 301) [Size: 185] [--> http://10.6.6.23/admin/]
/assets               (Status: 301) [Size: 185] [--> http://10.6.6.23/assets/]
/wp-login             (Status: 301) [Size: 185] [--> http://10.6.6.23/wp-login/]
/secret               (Status: 301) [Size: 185] [--> http://10.6.6.23/secret/]
/wp-admin             (Status: 301) [Size: 185] [--> http://10.6.6.23/wp-admin/]
<output omitted for brevity>
Finished
===============================================================
```

In **Example 9-42** different directories were enumerated by **gobuster** in a host configured with IP address 10.6.6.23. A wordlist called words.txt is used to perform the directory enumeration.

Tip

You can obtain numerous wordlists from the SecLists GitHub repository at **https://github.com/danielmiessler/SecLists**.

This repository is also cloned/available in WebSploit Labs under the /root/SecLists directory.

---

ffuf is another great tool written in Go. It's used for URL brute-forcing, fuzzing, and content discovery during the reconnaissance phase of a web application assessment. You can access the source code at **https://github.com/ffuf/ffuf**.

Example 9-46 demonstrates a high-level example of how to use **ffuf**.

Example 9-46 Using *ffuf* to Perform Directory Enumeration

Click here to view code image

```
┌──(root㉿websploit)-[~]
└─# ffuf -c -w words.txt -u http://10.6.6.23/FUZZ
       /'___\  /'___\           /'___\
      /\ \__/ /\ \__/  __  __  /\ \__/
      \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
       \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
        \ \_\   \ \_\  \ \____/  \ \_\
         \/_/    \/_/   \/___/    \/_/

_____________________________________________________
 :: Method           : GET
 :: URL              : http://10.6.6.23/FUZZ
 :: Wordlist         : FUZZ: /root/words.txt
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200,204,301,302,307,401,403,405,500
_____________________________________________________
[Status: 200, Size: 10351, Words: 2137, Lines: 138, Duration: 0ms]
    * FUZZ: # on at least 2 different hosts

<output omitted for brevity>

    * FUZZ: # This work is licensed under the Creative Commons
[Status: 301, Size: 185, Words: 6, Lines: 8, Duration: 0ms]
    * FUZZ: s
[Status: 301, Size: 185, Words: 6, Lines: 8, Duration: 0ms]
    * FUZZ: admin
[Status: 301, Size: 185, Words: 6, Lines: 8, Duration: 0ms]
    * FUZZ: assets
[Status: 301, Size: 185, Words: 6, Lines: 8, Duration: 0ms]
    * FUZZ: wp-login
[Status: 301, Size: 185, Words: 6, Lines: 8, Duration: 0ms]
```

```
    * FUZZ: secret
[Status: 301, Size: 185, Words: 6, Lines: 8, Duration: 0ms]
    * FUZZ: wp-admin
<output omitted for brevity>
```

The lines highlighted in **Example 9-46** show the discovered directories in the web application running on 10.6.6.23.

Tip

You can obtain additional examples of how to use **ffuf** at **https://becomingahacker.org/a-quick-guide-to-using-ffuf-with-burp-suite-713492f62242**.

The following tools also can be used to obtain additional details about a web application:

- **Wappalyzer** (**https://www.wappalyzer.com**) detects the content management systems, frameworks, and coding languages utilized on a website.
- **BuiltWith** (**https://builtwith.com**) serves as a tool that reveals the web technologies underlying a particular website.
- **StackShare** (**https://stackshare.io**) functions as a virtual hub where developers can divulge the technology stacks they employ, facilitating the gathering of data regarding your objective.
- **Retire.js** (**https://retirejs.github.io/retire.js**) identifies obsolete JavaScript libraries and Node.js packages in use.
- The **OWASP Zed Attack Proxy (ZAP)** is an open-source web application security testing tool used by developers, penetration testers, and security professionals to identify vulnerabilities in web applications.

ZAP is a dynamic application security testing (DAST) tool, meaning it tests running applications by simulating attacks and analyzing their responses. ZAP can act as a web application proxy and "man-in-the-middle" between the user's browser and the web application, intercepting and analyzing HTTP/HTTPS traffic. This tool allows users to inspect, modify, and forward requests/responses in real time.

ZAP also supports automated scanning and spidering (crawling) the web application. ZAP supports various authentication methods (form-based, HTTP-based) and can test different user roles within an application to ensure proper access control. ZAP also includes a fuzzer that sends unex-

pected or incorrect data to the application to test its resilience against malformed inputs.

ZAP can be integrated into CI/CD pipelines using its API or plug-ins for tools like Jenkins, allowing automated security testing during development. ZAP has a marketplace with numerous add-ons that extend its functionality, such as additional scanning rules or support for new technologies.

API Recon

Interactive API analysis involves actively navigating through the web application using both a browser and an API client. The goal is to familiarize yourself with all the potential interactive elements and scrutinize them closely. In essence, you will be investigating the web page, intercepting requests, searching for API links and documentation, and gaining insight into the underlying business logic.

Typically, the application should be evaluated from three angles: guests, authenticated users, and site administrators. Guests represent anonymous users, potentially exploring the site for the first time. If the platform mainly displays public information without necessitating user authentication, it might predominantly cater to guest users. Authenticated users, on the other hand, have undergone a registration procedure, acquiring a designated level of access. Administrators possess the rights to oversee and administer the API.

The initial step involves browsing the website and assessing it from these varied viewpoints. Here are a few pertinent questions for each user category:

- **Guest**: What would be the approach of a novice user toward this site? Is interaction with the API feasible for new users? Is the API documentation publicly accessible? What functionalities are available to this group?
- **Authenticated User**: What additional privileges are available upon authentication compared to guest access? Is there an option to upload files or navigate through new segments of the web application? What is the method of API utilization, and how does the application distinguish authenticated users?
- **Administrator**: What is the designated login area for administrators to manage the web app? What insights can be derived from the page source? Are there any remarks scattered across various pages? Which

programming languages are employed? Are there sections of the website currently under development or in the experimental stage?

Subsequently, you should adopt the perspective of a hacker to scrutinize the app by capturing the HTTP traffic using tools like Burp Suite. Interaction with the web app's search bar or authentication processes may involve API requests, which will be visible in Burp Suite.

Burp Suite is a popular and comprehensive web application security testing platform widely used by penetration testers, red teamers, and security professionals. It functions as an interception proxy, allowing users to inspect, modify, and manipulate HTTP and HTTPS traffic between a browser and a web server. The tool is available in both a free Community Edition and a more feature-rich Professional Edition, with the latter offering advanced automated scanning tools and integrations for continuous testing workflows.

Burp Suite includes several powerful tools that enable both manual and automated security testing. Key features include the Spider, which crawls web applications to discover endpoints; Intruder, which automates attacks on input fields to test for vulnerabilities; Repeater, which allows manual modification of requests for testing purposes; and Scanner, an automated tool for detecting common security flaws. Additionally, Burp Suite supports extensions through its BApp Store, enabling users to expand its functionality with custom plug-ins. Its versatility and ease of use make it a go-to tool for both small-scale testing and enterprise-level security assessments.

Encountering obstacles signifies the necessity to revisit findings from the initial phase scans operating in the background and to initiate the next phase: specialized scans. During this phase, you can hone your scans and employ tools tailored for your specific target. Unlike detection scans, which encompass a broader range, specialized scans should concentrate on particular API types, versions, the nature of the web application, discovered service versions, the protocol (HTTP or HTTPS) used by the app, active TCP ports, and other details uncovered from studying the business logic. For instance, discovering an API operating over an unconventional TCP port allows you to adjust your scanners to scrutinize that port meticulously. If the web application is developed using WordPress, you can verify the accessibility of the WordPress API by navigating to /wp-json/wp/v2. At this juncture, you ought to be familiar with the web application URLs and can commence brute-forcing uniform resource identifiers to uncover concealed directories and files. As these tools function, you can continu-

ally review the incoming results to conduct a more focused interactive analysis.

Subsequent segments will elaborate on the instruments and methods to be utilized across the active reconnaissance stages, encompassing detection scans using Nmap, interactive analysis with DevTools, and specialized scanning employing Burp Suite and OWASP ZAP.

Browser Developer Tools Are Sometimes Underrated

You can use browser developer tools to perform many tasks that can help you with reconnaissance. Chrome DevTools houses several often-overlooked tools for hacking web applications. The subsequent procedures will guide you in navigating through numerous lines of code adeptly, enabling you to identify sensitive data within page sources.

Start by accessing your target page, and then launch Chrome DevTools either by pressing **F12** or **Ctrl+Shift+I**. Modify the Chrome DevTools window dimensions to secure sufficient working space. Next, navigate to the Network tab and refresh the web page.

Subsequently, scrutinize for notable files (it's possible to encounter one named "API"). On finding JavaScript files that pique your interest, right-click them and select **Open** in the Sources Panel to explore their source code. As an alternative, click **XHR** to view the Ajax requests being executed.

Initiate a search for lines of JavaScript that might contain intriguing information. Focus your search on key phrases such as "API," "APIkey," "secret," and "password." This method might help you unearth an API buried nearly 4,200 lines deep within a script.

Additionally, the Memory tab in DevTools can be an invaluable resource, permitting you to capture an image of the memory heap distribution. It's not uncommon to find static JavaScript files containing vast amounts of information and extensive lines of code. This can sometimes make it challenging to ascertain the exact manner in which a web app interacts with an API. In such cases, you can employ the Memory panel to monitor the resource utilization patterns of the web application during its interaction with the API.

To use this feature, select the Memory tab. Within the Select Profiling Type section, opt for **Heap Snapshot**. Following this, in the Select

JavaScript VM Instance section, pinpoint the target for evaluation. Then finally, initiate the process by clicking the **Take Snapshot** button.

If you are familiar with targeting web applications, searching for vulnerabilities in APIs should feel somewhat familiar. The key distinction here is that you won't have the typical graphical user interface indicators like search bars, login fields, and file upload buttons. Instead, API hacking revolves around the backend operations that underlie these GUI elements, specifically focusing on GET requests with query parameters and most POST/PUT/UPDATE/DELETE requests.

Before you start crafting requests to interact with an API, you must gain an understanding of its endpoints, request parameters, required headers, authentication protocols, and administrative functionalities. Documentation typically serves as your guide to these elements. Consequently, to excel as an API hacker, you need to be proficient in reading and utilizing API documentation and, ideally, know how to locate it. Additionally, if you can access an API specification, you can directly import it into tools like Postman to automate the request crafting process.

In scenarios where you are conducting a black-box API test and genuine documentation is unavailable, you'll need to reverse-engineer the API requests independently. This process entails thoroughly testing the API, experimenting with different inputs to discover endpoints, parameters, and header prerequisites, essentially mapping out the API and comprehending its capabilities.

Tip

Remember that API documentation serves as just the initial step. It's unwise to blindly trust that the documentation is entirely accurate, up to date, or exhaustive in detailing all aspects of the endpoints. Always validate by testing for methods, endpoints, and parameters that might not be documented. In this context, a healthy dose of skepticism and thorough verification are essential.

In the next chapter (**Chapter 10**), we will dive deep into many additional methods while exploiting vulnerabilities in web applications and APIs.

Communicating Your Findings and Creating Effective Bug Bounty Reports

Effectively communicating findings in a bug bounty program to organizations is a critical step in your ethical hacker role. Your ability to convey discovered vulnerabilities and potential threats clearly and professionally can significantly impact the organization's security posture.

Your role as an ethical hacker extends beyond the identification of vulnerabilities; it includes the crucial task of communicating these findings to organizations in a manner that facilitates prompt remediation and strengthens security measures. Effective communication is the bridge that connects your technical expertise to actionable improvements in an organization's cybersecurity. The following are key steps to make sure that you communicate your findings effectively:

1. **Document Your Findings Thoroughly**
    1. Before communicating anything to the organization, create a detailed report of your findings. Include comprehensive descriptions of vulnerabilities, their potential impact, and clear evidence or proof of concept.
    2. Organize your findings logically and categorize them by severity to help the organization prioritize remediation efforts.
2. **Use Clear and Nontechnical Language**
    1. Remember that not all stakeholders may have the same technical expertise as you. Use plain language to describe the issues and their implications.
    2. Avoid jargon or overly technical terminology, and provide explanations when necessary.
3. **Offer Remediation Suggestions**
    1. Don't just point out the problems; propose practical solutions or mitigation strategies whenever possible. This demonstrates your commitment to helping the organization improve its security posture.
    2. Clearly outline steps or recommendations for addressing each vulnerability.
4. **Prioritize and Contextualize**
    1. Highlight the most critical vulnerabilities first, especially those that pose an immediate risk. Doing so helps the organization focus on addressing the most pressing issues.
    2. Provide context by explaining how each vulnerability could be exploited and the potential impact on the organization.

3. Include evidence. Attach any relevant evidence, such as screenshots, log files, or proof-of-concept code, to substantiate your findings. Taking this step adds credibility to your report and helps the organization understand the issue better.

5. **Maintain Professionalism**
    1. Maintain a professional and respectful tone throughout your communication. Avoid accusations or finger-pointing because your goal is to collaborate with the organization, not antagonize them.
    2. Be prepared to answer questions or provide clarifications promptly and courteously.
6. **Follow Secure Communication Practices**
    1. Ensure that you use secure channels for communication, especially when sharing sensitive information. Encrypt emails or use secure messaging platforms if necessary.
    2. Respect the organization's disclosure policies and adhere to any nondisclosure agreements (NDAs) you've entered into.
7. **Establish a Channel for Follow-Up**
    1. Provide contact information and a designated channel for the organization to reach out to you with questions or updates on remediation progress.
    2. Be available for discussions and clarifications to facilitate a collaborative approach.
8. **Stay Engaged and Supportive**

    Continue to engage with the organization throughout the remediation process. Offer assistance if needed, and remain committed to helping them enhance their security measures.

Once the organization has remediated the vulnerabilities, request confirmation and evidence of the fixes. Acknowledge and confirm closure of the issues in your report.

In this chapter, you learned details about bug bounty programs. You've gained insights into the fundamental concepts underpinning bug bounty programs and their role in today's environment. You now know the different types of bug bounty programs. Beyond technical expertise, you've delved into the realm of legal and ethical considerations. This chapter has emphasized the importance of proper scope and adhering to legal boundaries while engaging in bug bounty programs and conducting reconnaissance activities.

As technology advances, the integration of AI technologies has become a significant aspect of cybersecurity. You now have an understanding of how AI can enhance the efficiency and effectiveness of reconnaissance

processes, keeping you at the forefront of cybersecurity advancements. You will learn more about how to use AI for bug bounty operations in **Chapter 11**.

Test Your Skills

Multiple-Choice Questions

1. What is the primary objective of attack surface management (ASM)?

1. Identifying and mitigating vulnerabilities before they can be exploited
2. Offering monetary rewards to individuals who find vulnerabilities
3. Establishing a formal process to report vulnerabilities
4. Encouraging community participation and collaboration

2. Who primarily conducts attack surface management?

1. External community of ethical hackers
2. Internal security teams supplemented by external consultants
3. General public
4. Individuals who find vulnerabilities incidentally

3. What distinguishes bug bounty programs from vulnerability disclosure programs (VDPs) in terms of incentives?

1. Both offer monetary rewards.
2. Bug bounty programs generally offer monetary or other material rewards.
3. VDPs offer higher monetary rewards.
4. There are no incentives in bug bounty programs.

4. Which program is typically more focused, defining specific targets and rules?

1. Attack surface management
2. Vulnerability disclosure program (VDP)
3. Bug bounty programs
4. Both b and c

5. What is a key feature of vulnerability disclosure programs (VDPs)?

1. Offering monetary rewards
2. Creating a formalized process for reporting vulnerabilities
3. Focusing only on specific applications or systems

4. Being conducted by external communities of hackers

6. What kind of legal protection does a vulnerability disclosure program offer?

1. No legal protection
2. Protection against legal repercussions when guidelines are followed
3. Unconditional legal protection
4. Legal protection only for internal employees

7. Who are the typical participants in bug bounty programs?

1. Internal security teams only
2. Ethical hackers and security researchers from the external community
3. External consultants only
4. General public without any restrictions

8. What is the approach of attack surface management regarding frequency and adjustments?

1. Periodic with defined periods
2. Reactive with spontaneous assessments
3. Ongoing with regular assessments and adjustments
4. Only initiated after a security incident

9. Which of the following is a characteristic of a proactive security approach?

1. Vulnerability disclosure program (VDP)
2. Bug bounty program
3. Attack surface management
4. Reactive incident response

10. In the context of bug bounty programs, what does scope refer to?

1. The range of rewards offered
2. The range or domain within which researchers can find and report vulnerabilities
3. The list of participants involved in the program
4. The legal frameworks governing the program

11. Which of the following tools is commonly used for directory and file brute-forcing?

1. Amass
2. Maltego
3. Gobuster
4. Ffuf

12. What is the primary function of Ffuf?

1. Network mapping
2. Web fuzzing, file, and directory enumeration
3. Open-source intelligence gathering
4. Certificate management

13. Certificate transparency primarily helps in detecting

1. Slow network connections.
2. Fraudulent SSL certificates.
3. Directory structures on web servers.
4. API keys in source code.

14. What is the main function of the tool Amass?

1. Graphic representation of programming structures
2. Advanced open-source intelligence gathering
3. API hacking reconnaissance by evaluating application web content at scale
4. Network defense

15. Maltego is primarily used for

1. Creating multiple-choice questions.
2. OSINT.
3. Directory brute-forcing.
4. Fast web fuzzing.

Exercises

Exercise 9.1: AI-Driven Recon Using Gorilla

Task: Become familiar with generative AI tools and LLMs such as Gorilla.

In this chapter, you learned about how the University of California, Berkeley, and Microsoft introduced Gorilla, a refined Llama-based model that apparently outperforms GPT-4 in crafting API calls. One of Gorilla's standout features is its integration with a document retriever. This syn-

ergy empowers Gorilla to seamlessly adjust to modifications in documents during testing. Gorilla can significantly reduce the hallucination problem, a frequent hiccup encountered when directly prompting LLMs.

Steps:

1. Refer to this blog post: **https://becomingahacker.org/using-gorilla-pioneering-api-interactions-in-large-language-models-for-cybersecurity-operations-252ce018be6b**.
2. Deploy WebSploit Labs by following the instructions at **websploit.org**.
3. Install gorilla-cli in WebSploit Labs by using the **pip3 install gorilla-cli** command.
4. Research other common OSINT and active reconnaissance tools in the **hackerrepo.org** GitHub repository.
5. Use Gorilla to interact with different tools to perform passive and active reconnaissance.

Exercise 9.2: Deploy the OWASP completely ridiculous API (crAPI)

Task: Deploy the OWASP intentionally vulnerable API called crAPI. In the next chapter (**Chapter 10**), you will learn how to hack this API.

Steps:

1. Access the crAPI documentation and installation information from **https://github.com/OWASP/crAPI**.
2. To deploy in a Linux system, use the following commands:
   Click here to view code image

```
curl -o docker-compose.yml
https://raw.githubusercontent.com/OWASP/crAPI/main/deploy/docker/
docker-compose.yml
docker-compose pull
docker-compose -f docker-compose.yml --compatibility up -d
```

Exercise 9.3: API Documentation Standards and Specifications

Task: Become familiar with API documentation standards and related resources.

1. **OpenAPI (Swagger)**: You can find information about OpenAPI on the OpenAPI Initiative website at **https://www.openapis.org**.
2. **API Blueprint**: Learn about API Blueprint on the API Blueprint website at **https://apiblueprint.org**. You can find tools like Aglio on

GitHub.

3. **RAML (RESTful API Modeling Language)**: You can find information about RAML on the RAML website at **https://raml.org**. Tools like RAML2HTML and API Designer are available on GitHub.
4. **GraphQL Documentation**: You can find GraphQL documentation at **https://graphql.org/learn**. Learn about Apollo Server at **https://www.apollographql.com/docs/apollo-server**.
5. **WSDL (Web Services Description Language)**: WSDL is commonly used for SOAP-based web services. It defines the structure and methods of the web service, including input and output data types. SOAP-based services often generate documentation automatically from the WSDL file. You can find information about WSDL in the W3C documentation at **https://www.w3.org/TR/wsdl/**.
6. **Postman Collections**: Postman allows developers to create collections of API requests and responses. These collections can be exported in JSON format and shared as documentation. Postman also offers a feature called Postman Docs for generating documentation directly from collections. You can explore Postman collections and documentation features on the Postman website at **https://www.postman.com**.

Exercise 9.4: API Enumeration

Task: Your job is to enumerate the available endpoints without making any requests yet. List all the endpoints you can find based on the provided documentation. After you've listed the endpoints, cross-reference your findings with the actual API documentation to check whether you've correctly enumerated all endpoints.

Steps:

1. Select a public API with multiple endpoints and HTTP methods (GET, POST, PUT, DELETE, and so on).
2. Use the same method to enumerate endpoints in crAPI.

You will learn additional tips and tricks on how to find vulnerabilities in web applications and APIs in the next chapter.